# Lab 03

## Working with Git History, Stashing, and Reverting Commits

**Name:** Saba Gull

**Registration No:** 2023-BSCS-074

**Course Code:** CS-649 (Cloud DevOps)

**Section:** A

**GitHub Username:** sabagu003-ui

## Repository Links

**Practice repository (commits):** https://github.com/sabagu003-ui/DevOps_Lab3_Practice

**Submission repository:** https://github.com/sabagu003-ui/DevOps

**Forked Gitea repository:** https://github.com/sabagu003-ui/gitea

**GitHub Pages repository:** https://github.com/sabagu003-ui/sabagu003-ui.github.io

**Live portfolio site:** https://sabagu003-ui.github.io

## Task 1: Handling Local and Remote Commit Conflicts (Pull vs Pull --rebase)

**Step 1:** Edited README.md directly on GitHub and committed the change (Remote update to README).

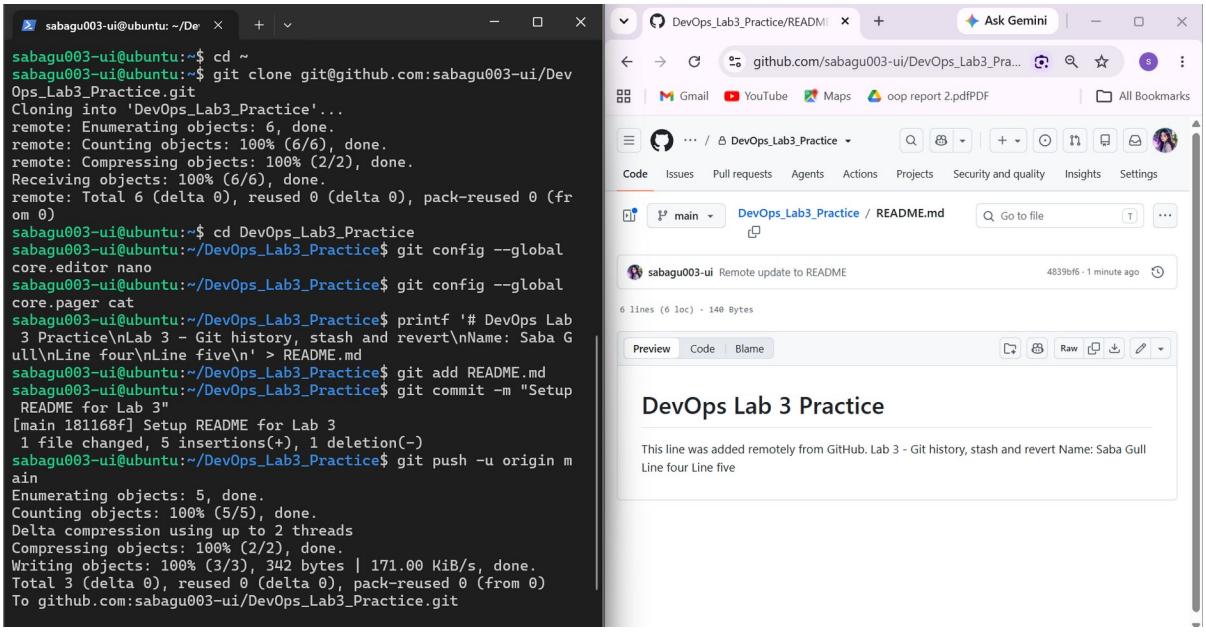



*Figure 1.1: Edited README.md directly on GitHub and committed the change (Remote update to README) (remote_edit.png)*

**Step 2:** Edited README.md locally and committed it (Local update to README).

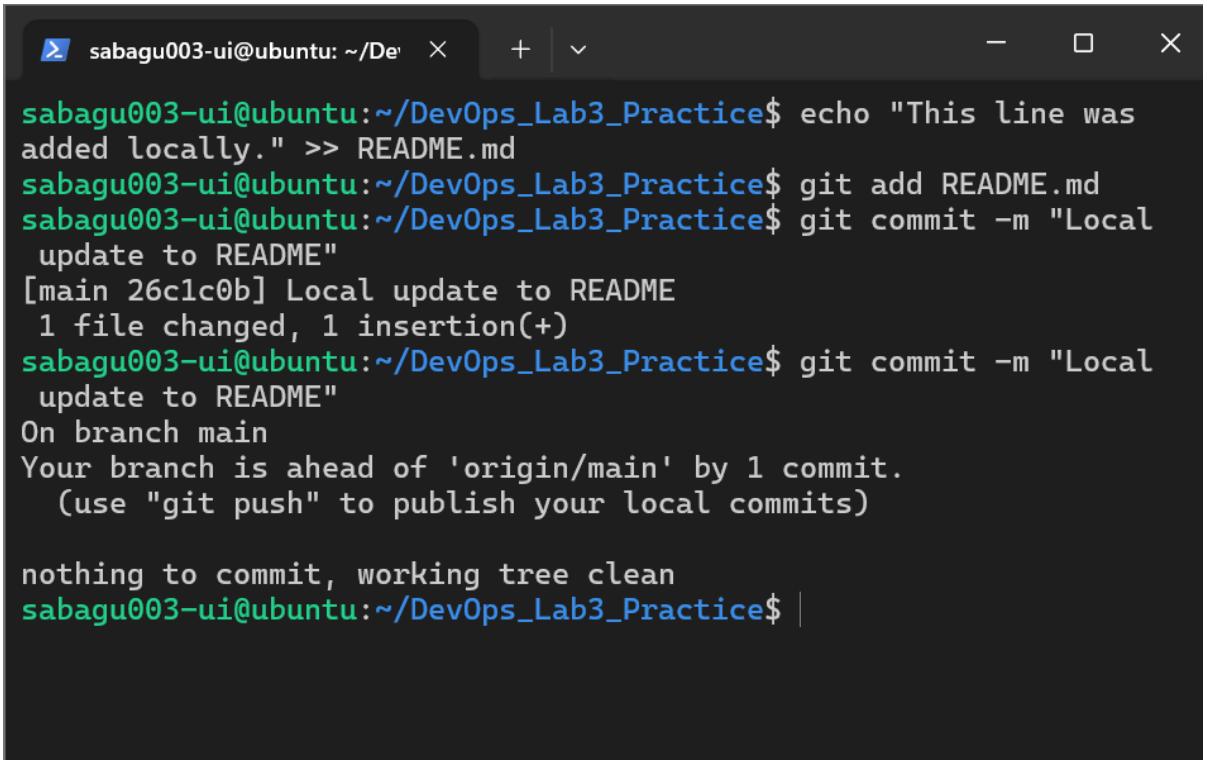



*Figure 1.2: Edited README.md locally and committed it (Local update to README) (local_commit.png)*

**Step 3:** Tried git push origin main. The push was rejected because the remote has new commits (fetch first).

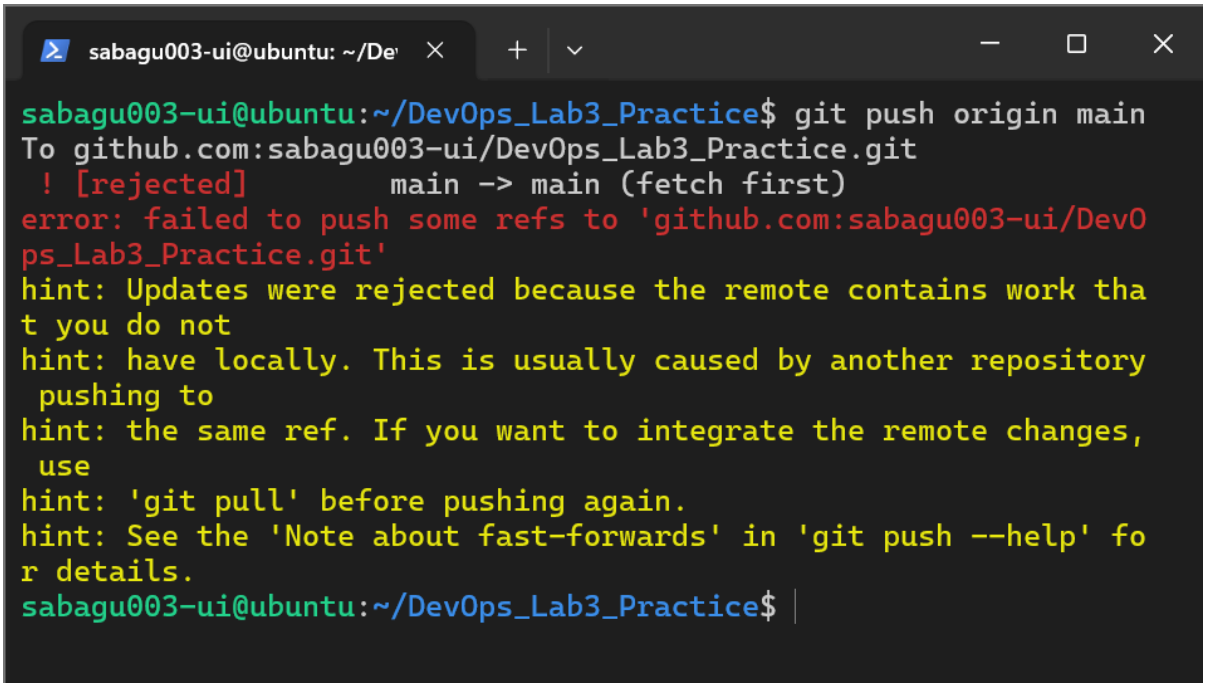



*Figure 1.3: Tried git push origin main (push_error.png)*

**Step 4:** Ran git pull --no-rebase origin main. Git merged the remote and local commits.

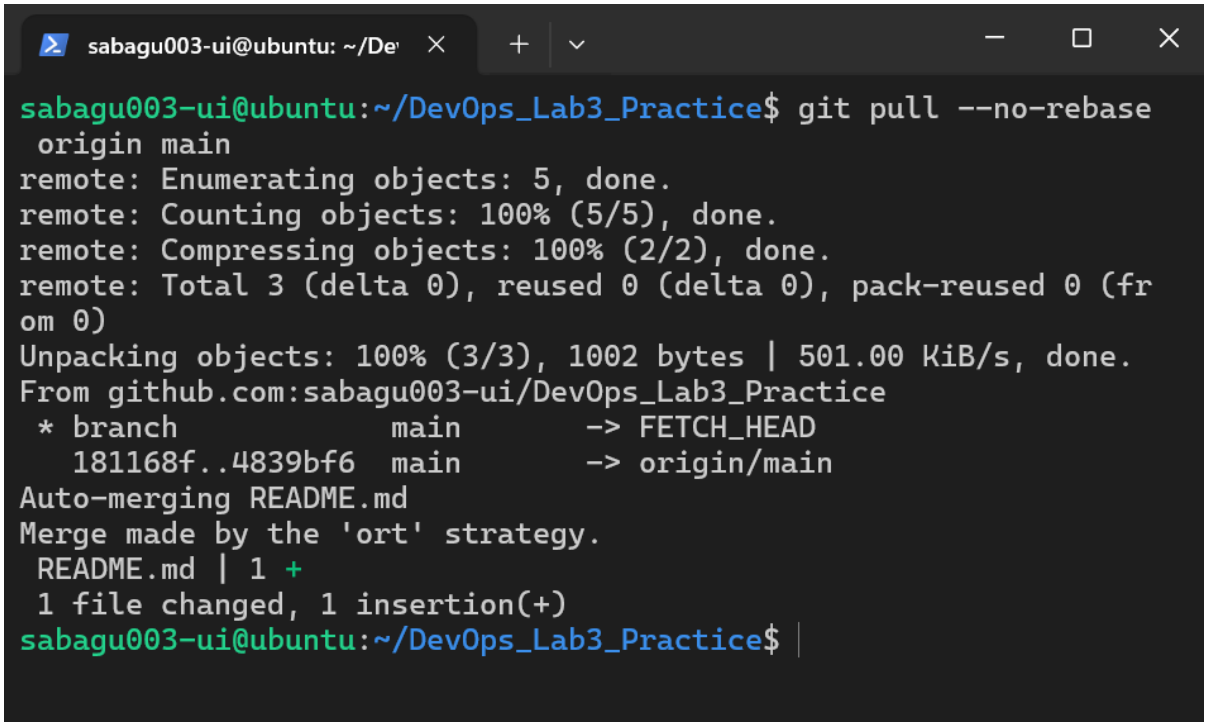



*Figure 1.4: Ran git pull --no-rebase origin main (merge_commit.png)*

**Step 5:** Pushed again with git push -u origin main after the merge.

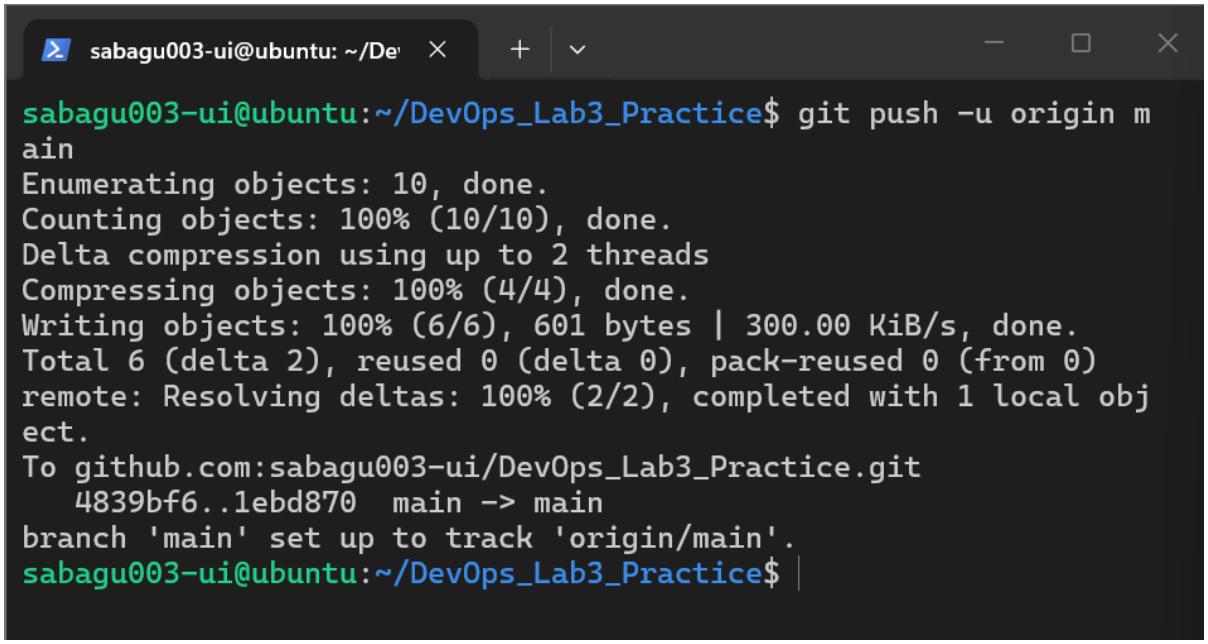



*Figure 1.5: Pushed again with git push -u origin main after the merge (push_after_merge.png)*

**Step 6:** Made another remote and local change, then ran git pull --rebase origin main. The local commit was replayed on top of the remote commits (linear history).

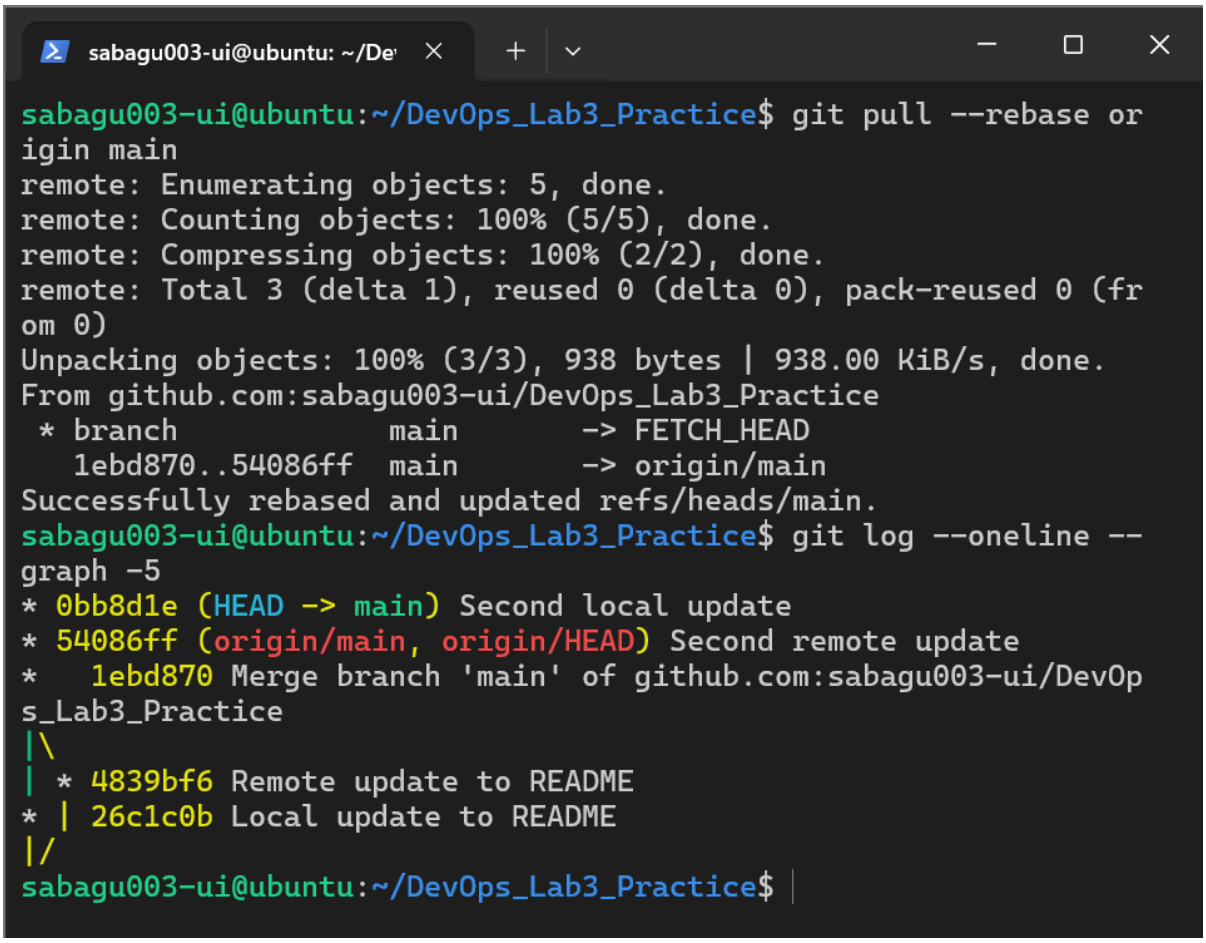



*Figure 1.6: Made another remote and local change, then ran git pull --rebase origin main (rebase_pull.png)*

**Step 7:** Pushed again after the rebase.

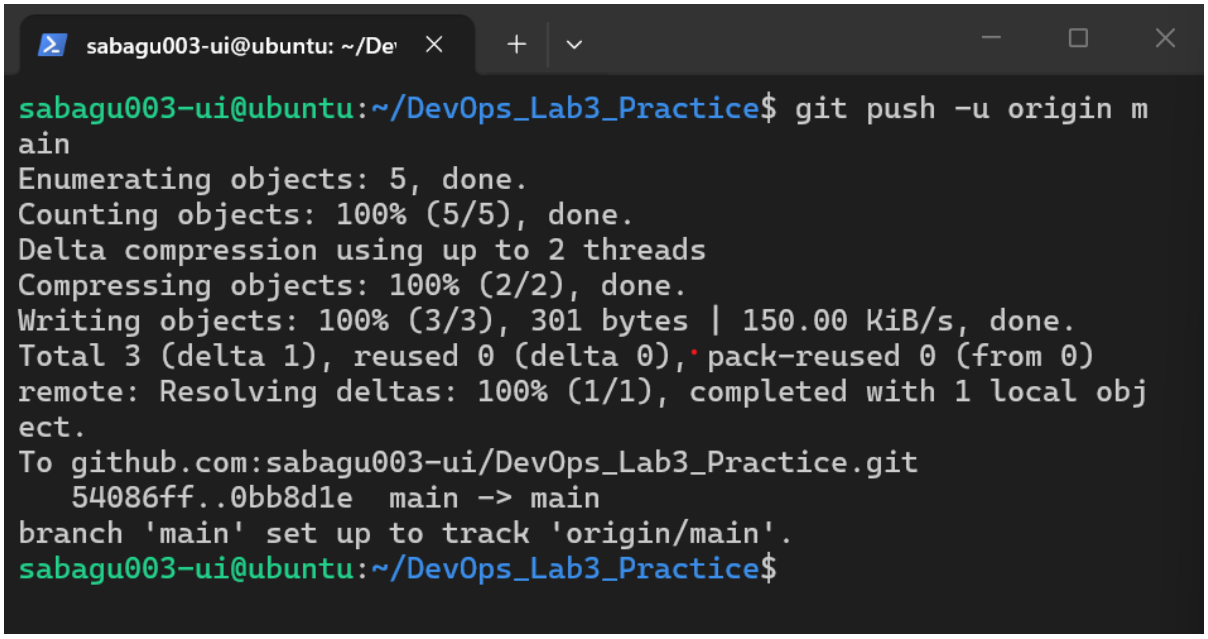



*Figure 1.7: Pushed again after the rebase (push_after_rebase.png)*

## Task 2: Creating and Resolving Merge Conflicts Manually

**Step 1:** Changed an existing line of README.md on GitHub and committed it.

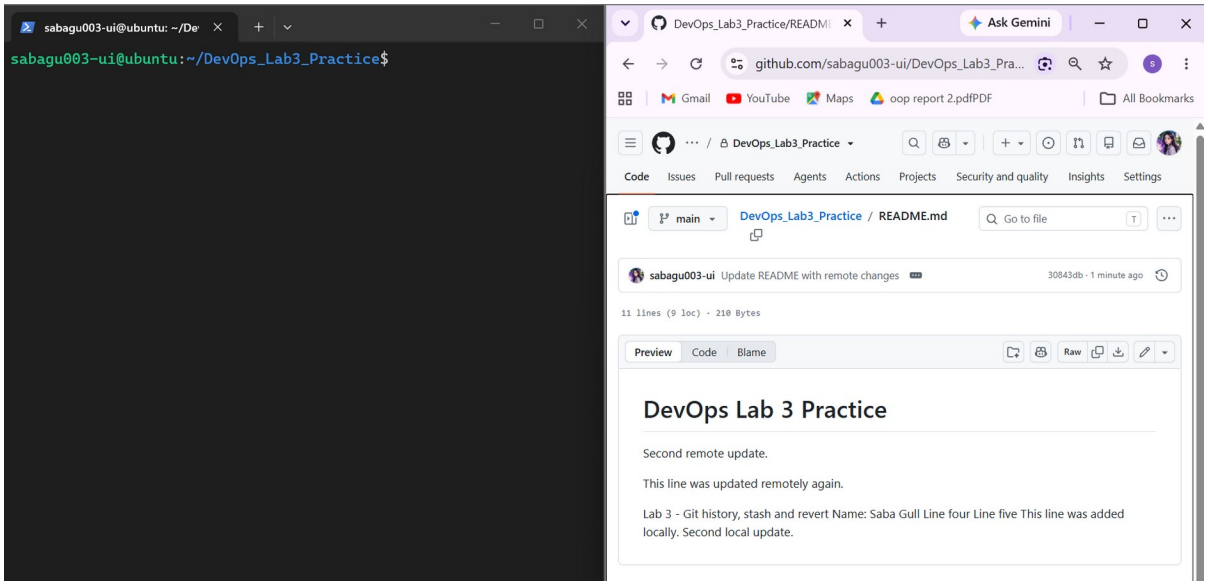



*Figure 2.1: Changed an existing line of README.md on GitHub and committed it (remote_conflict_edit.png)*

**Step 2:** Changed the same line locally in a different way (shown with git diff).

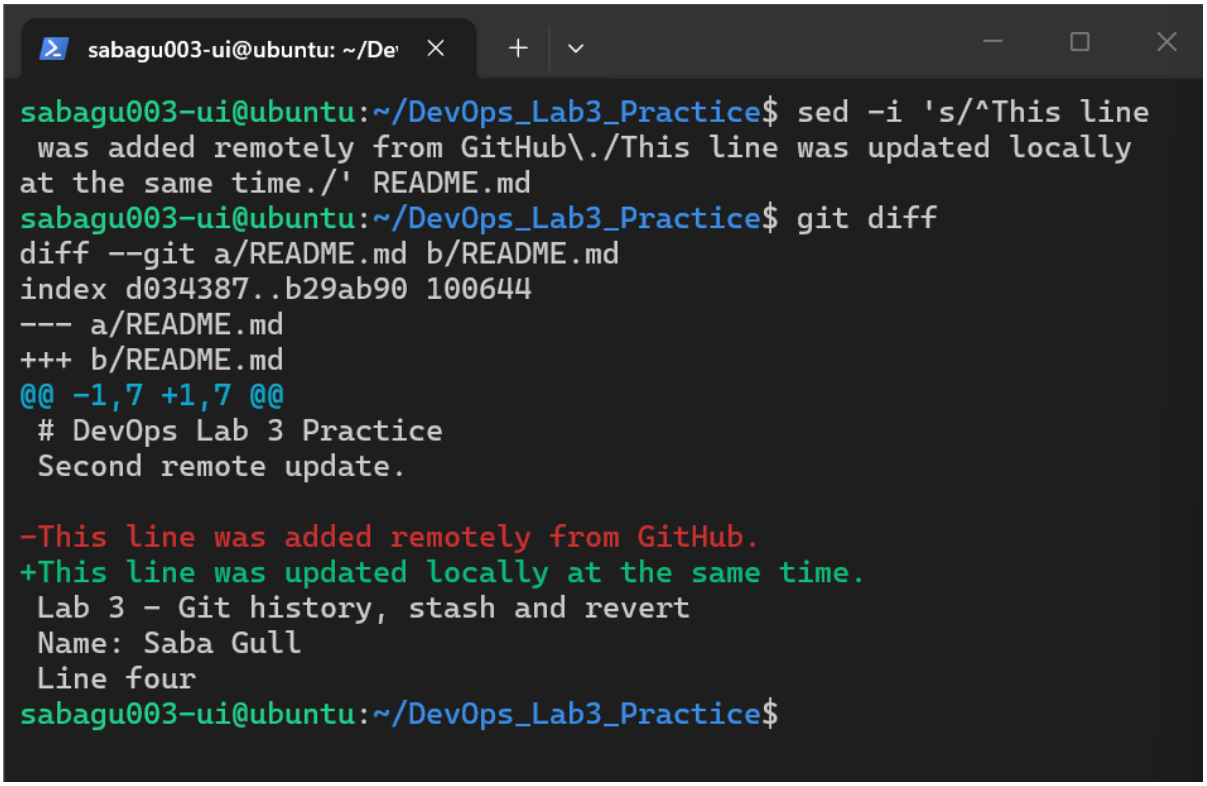



*Figure 2.2: Changed the same line locally in a different way (shown with git diff) (local_conflict_edit.png)*

**Step 3:** Committed the conflicting local change.

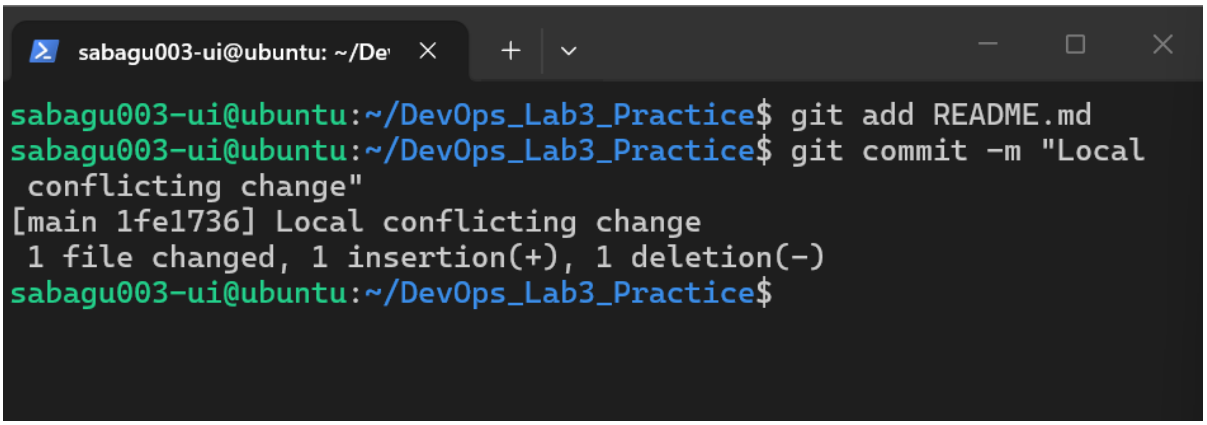



*Figure 2.3: Committed the conflicting local change (local_conflict_commit.png)*

**Step 4:** Push was rejected because the remote has conflicting changes.

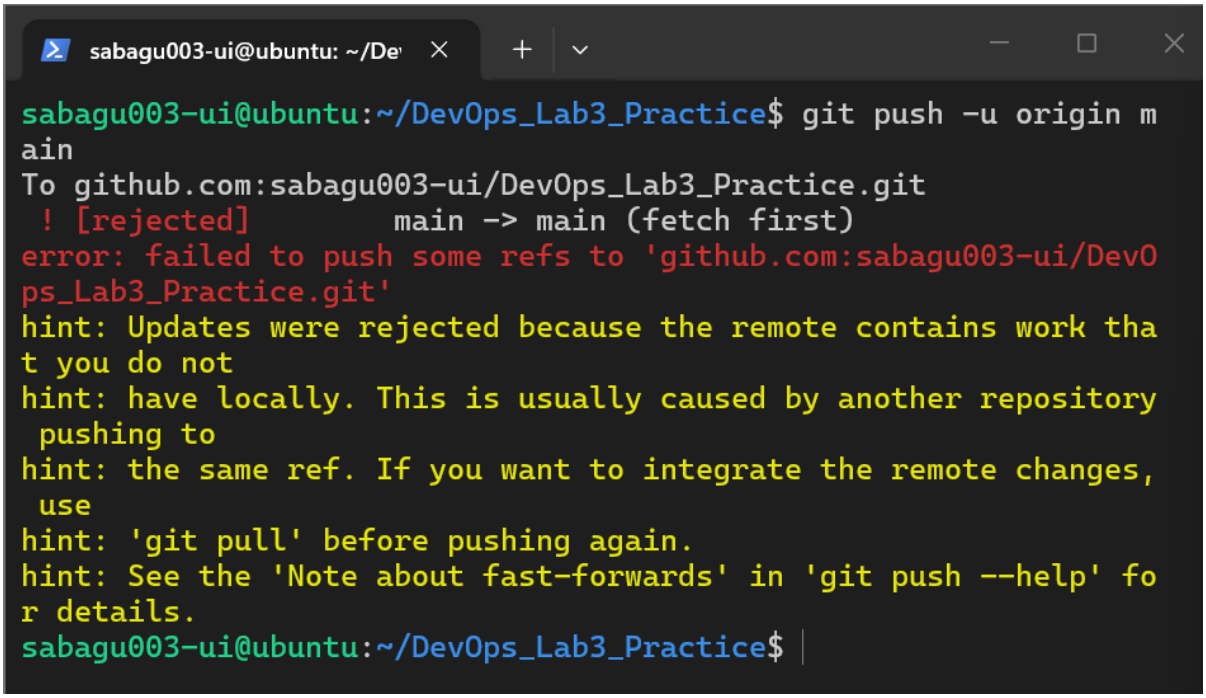



*Figure 2.4: Push was rejected because the remote has conflicting changes (conflict_push_error.png)*

**Step 5:** Ran git pull --rebase origin main. Git paused the rebase and reported a merge conflict in README.md.

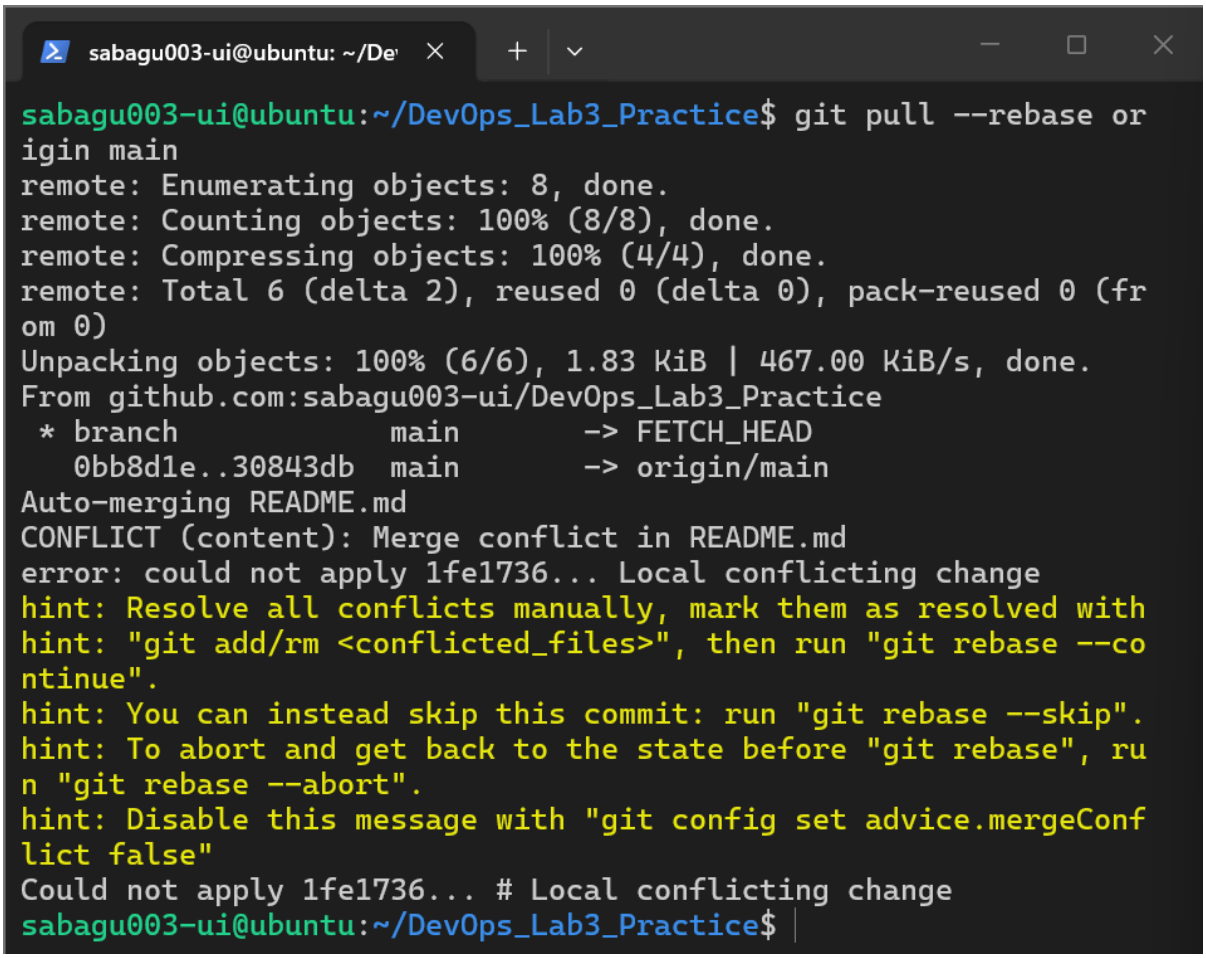



*Figure 2.5: Ran git pull --rebase origin main (conflict_message.png)*

**Step 6:** Edited README.md, removed the conflict markers and kept both changes.

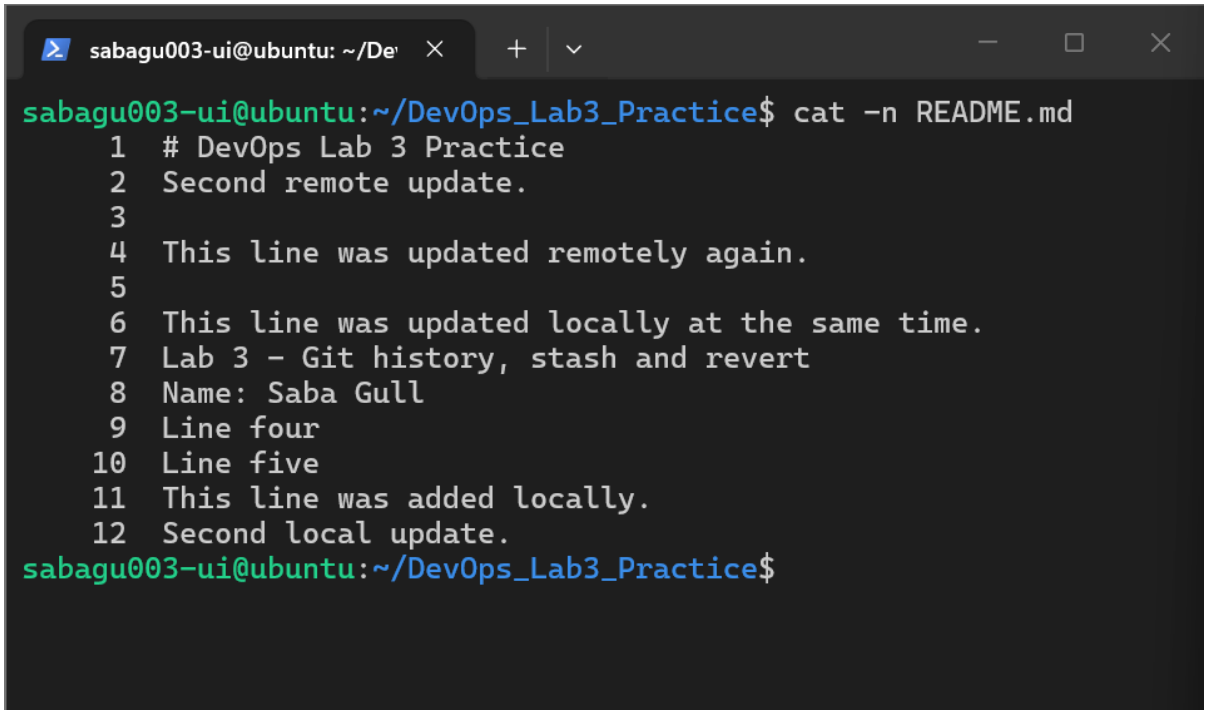



*Figure 2.6: Edited README.md, removed the conflict markers and kept both changes (resolved_readme.png)*

**Step 7:** Marked the conflict as resolved with git add and continued with git rebase --continue.

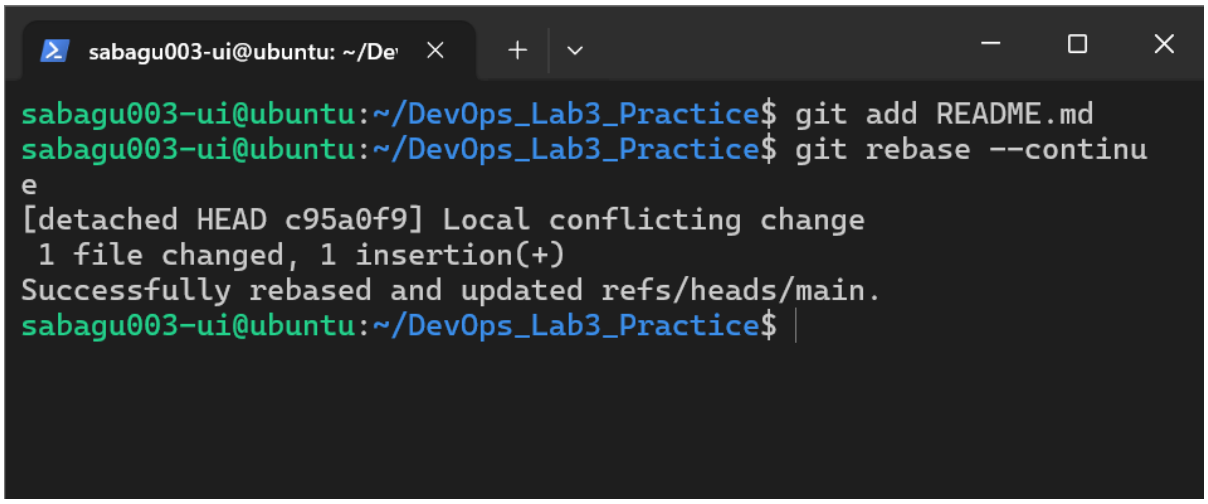



*Figure 2.7: Marked the conflict as resolved with git add and continued with git rebase --continue (rebase_continue.png)*

**Step 8:** Pushed the resolved changes to GitHub.

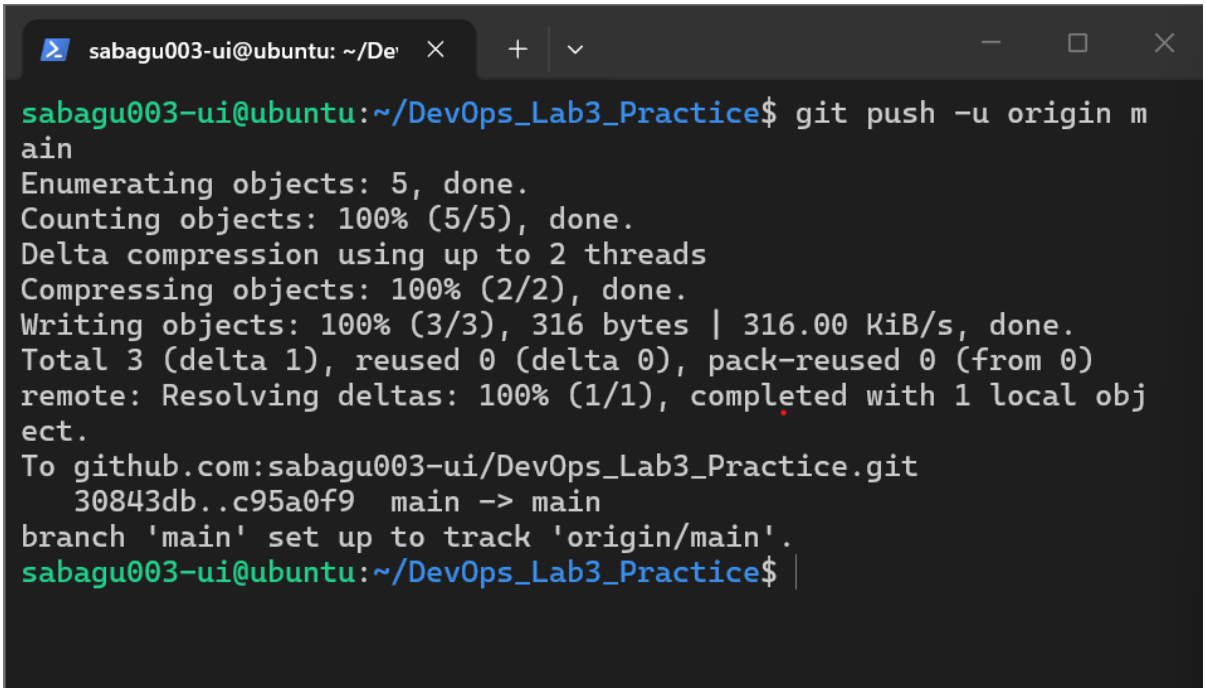



*Figure 2.8: Pushed the resolved changes to GitHub (push_after_resolve.png)*

## Task 3: Managing Ignored Files with .gitignore and Removing Tracked Files

**Step 1:** Created the textfiles directory with three files, committed and pushed it.

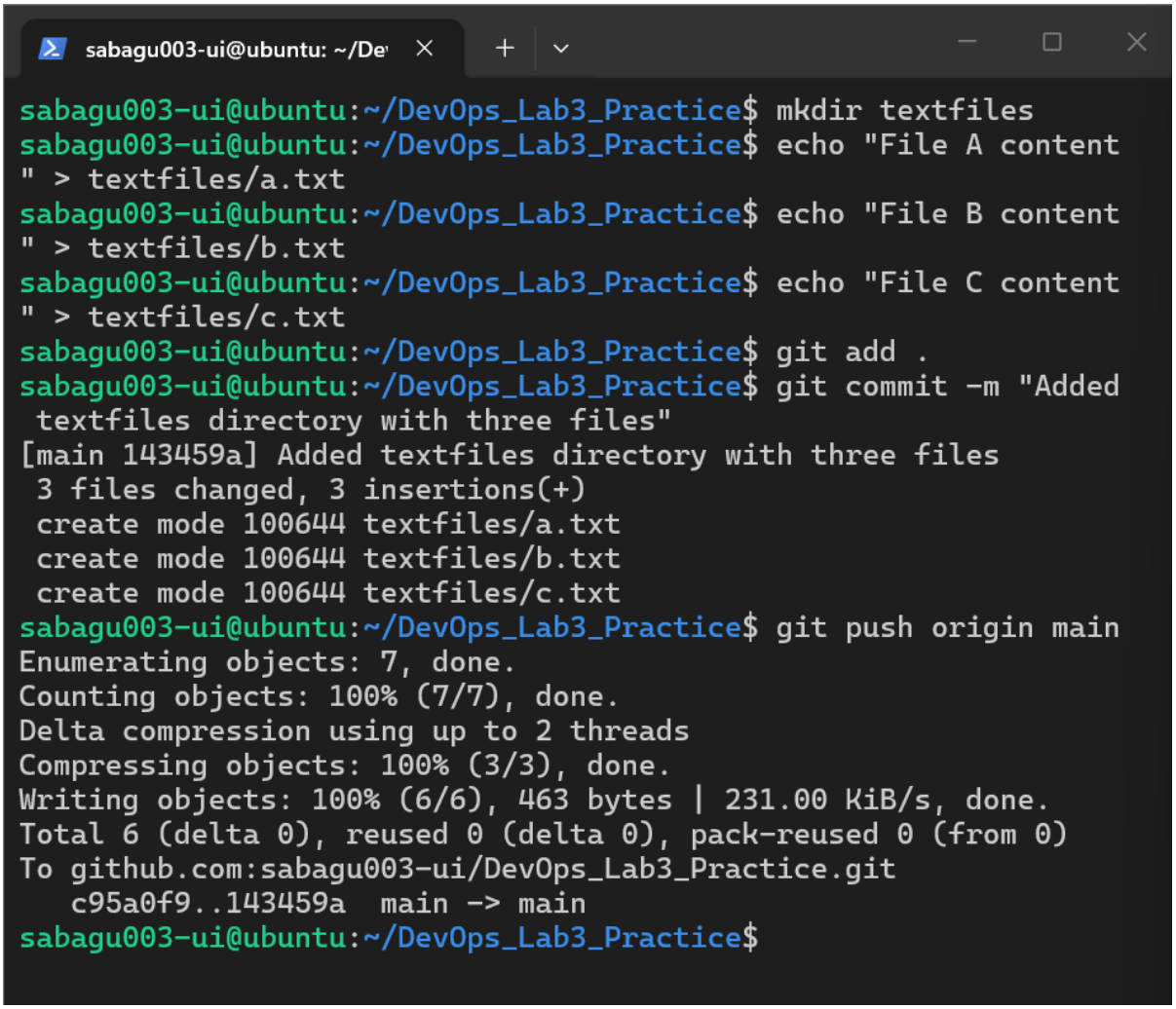



*Figure 3.1: Created the textfiles directory with three files, committed and pushed it (push_textfiles.png)*

**Step 2:** Created .gitignore with textfiles/*, committed and pushed it.

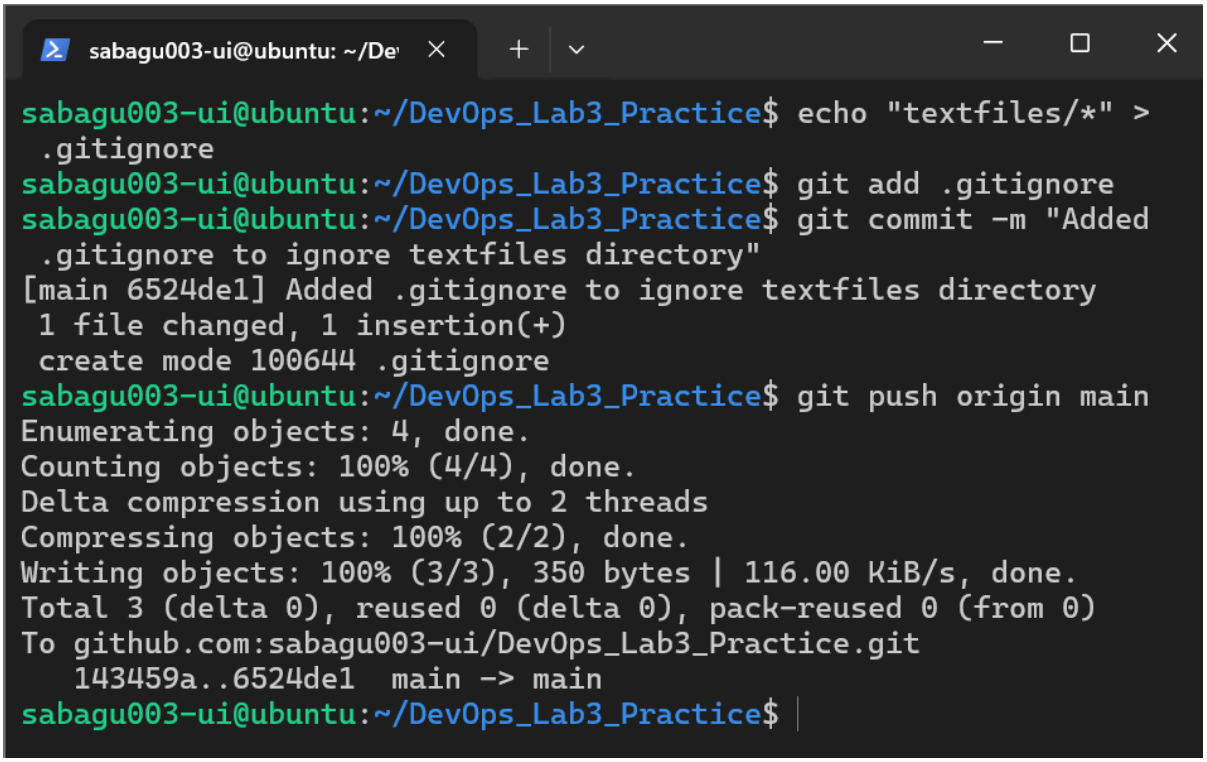



*Figure 3.2: Created .gitignore with textfiles/*, committed and pushed it (gitignore_push.png)*

**Step 3:** The textfiles directory is still visible on GitHub because it was already tracked.

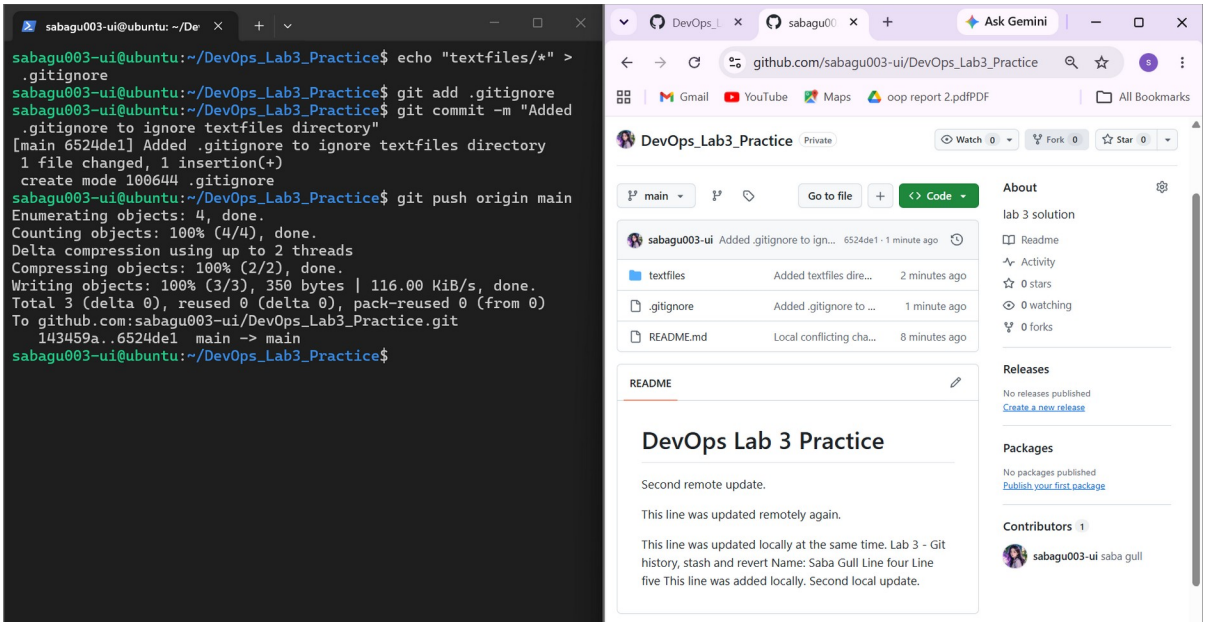



*Figure 3.3: The textfiles directory is still visible on GitHub because it was already tracked (repo_still_has_textfiles.png)*

**Step 4:** Removed the directory from tracking with git rm -r --cached textfiles, committed and pushed. The files still exist locally.

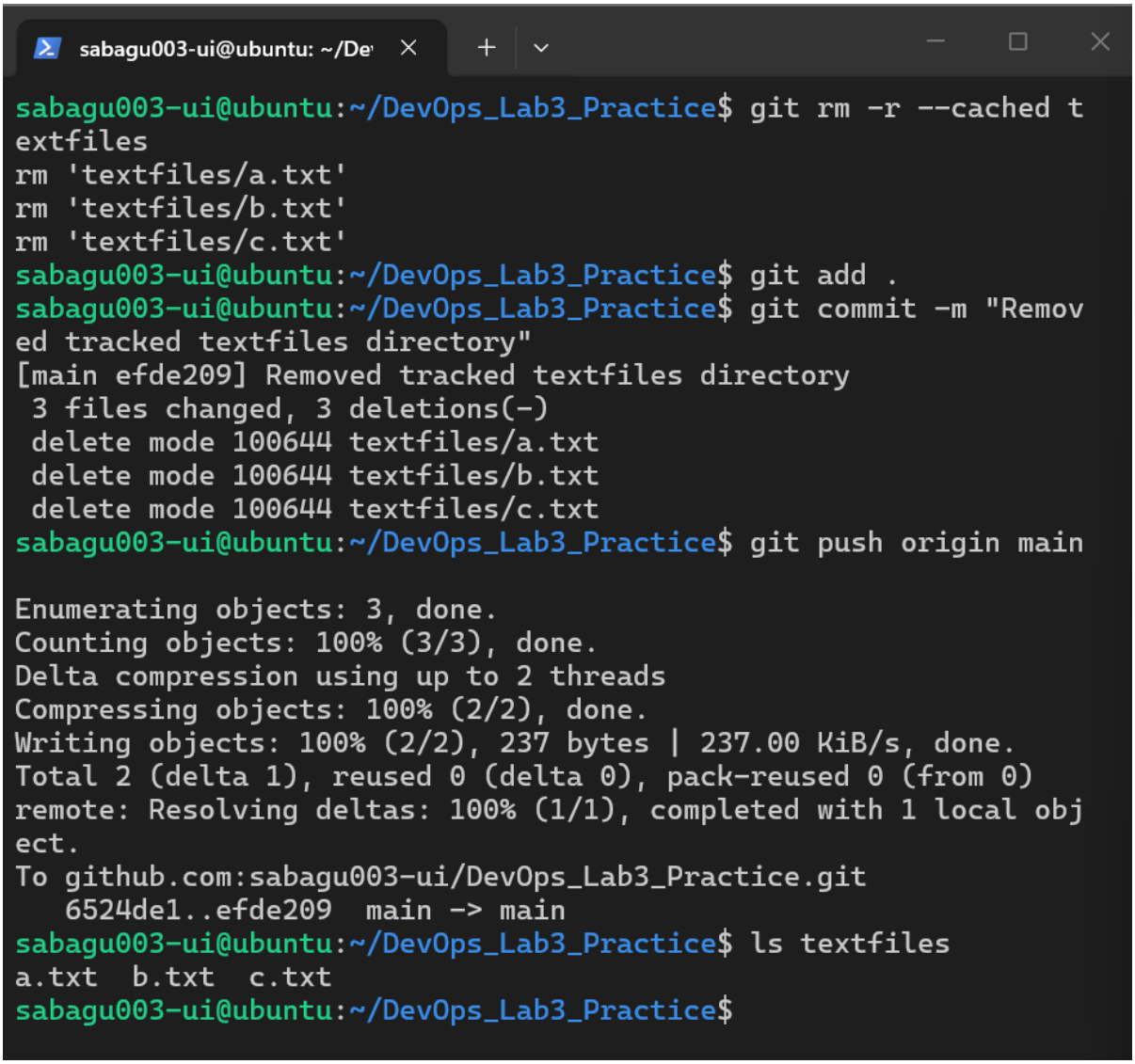



*Figure 3.4: Removed the directory from tracking with git rm -r --cached textfiles, committed and pushed (rm_cached_push.png)*

**Step 5:** The textfiles directory is now removed from the GitHub repository.

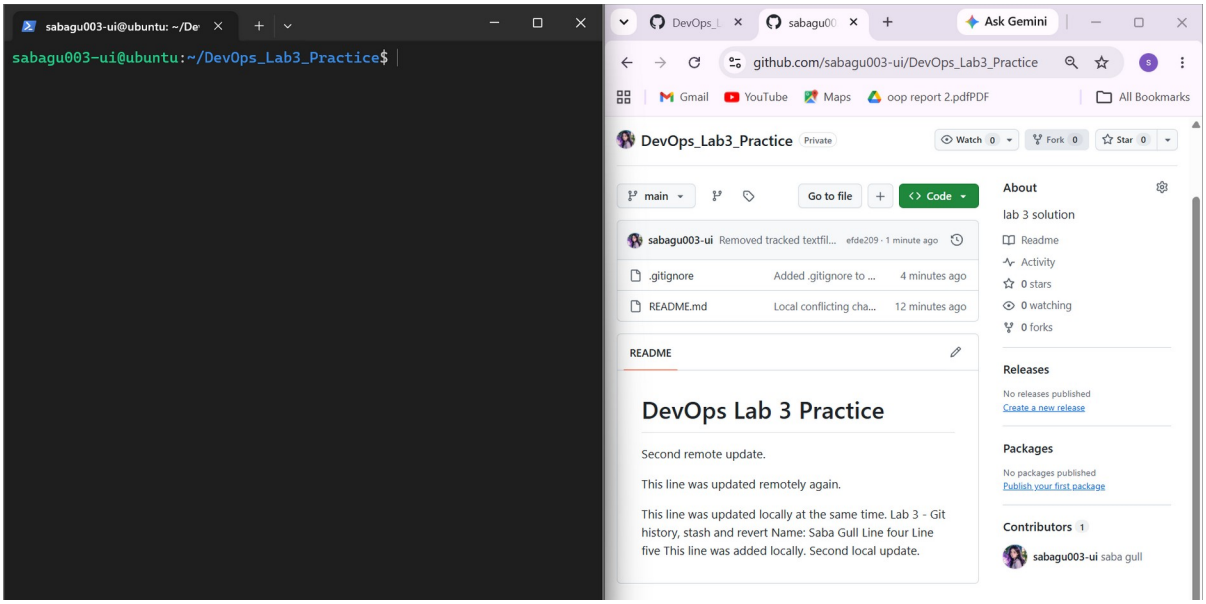



*Figure 3.5: The textfiles directory is now removed from the GitHub repository (repo_textfiles_removed.png)*

## Task 4: Create Temporary Changes and Use git stash

**Step 1:** Created feature-branch, modified README.md and committed the change.

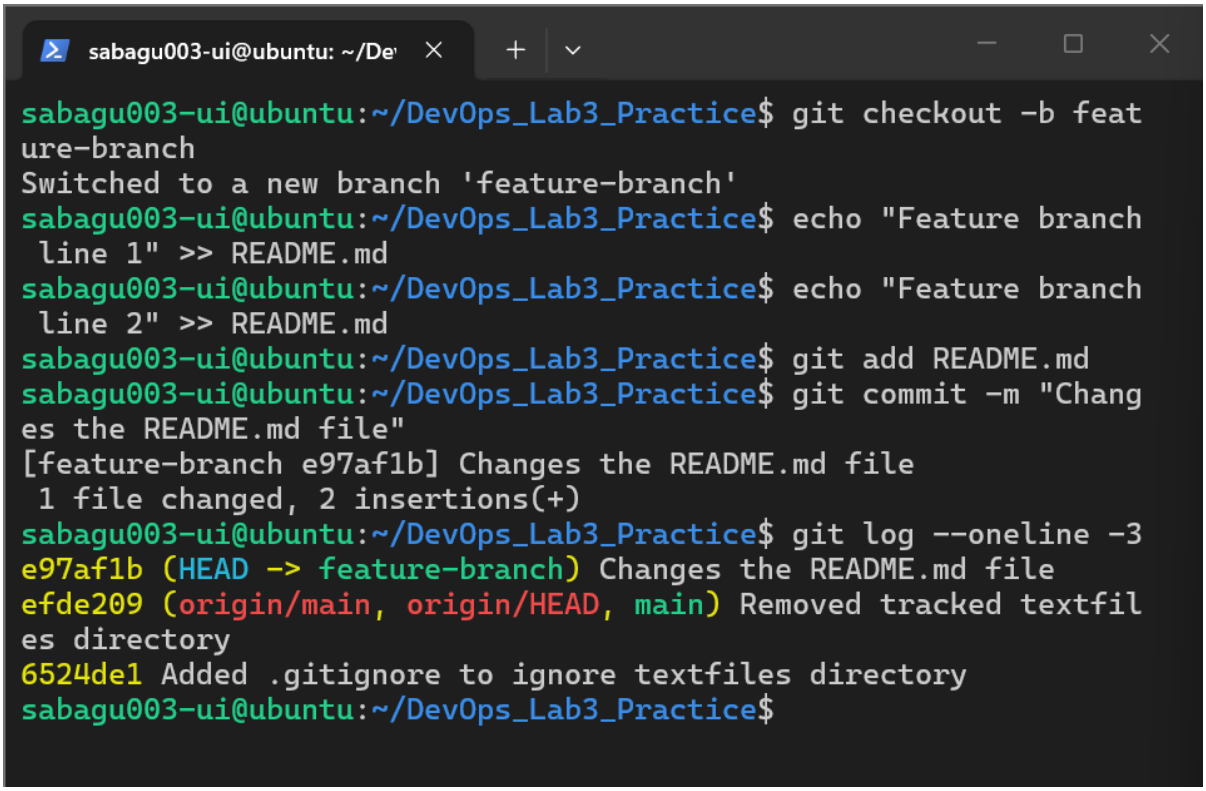



*Figure 4.1: Created feature-branch, modified README.md and committed the change (modified_readme.png)*

**Step 2:** Added uncommitted changes and tried git checkout main. Git refused because local changes would be overwritten.

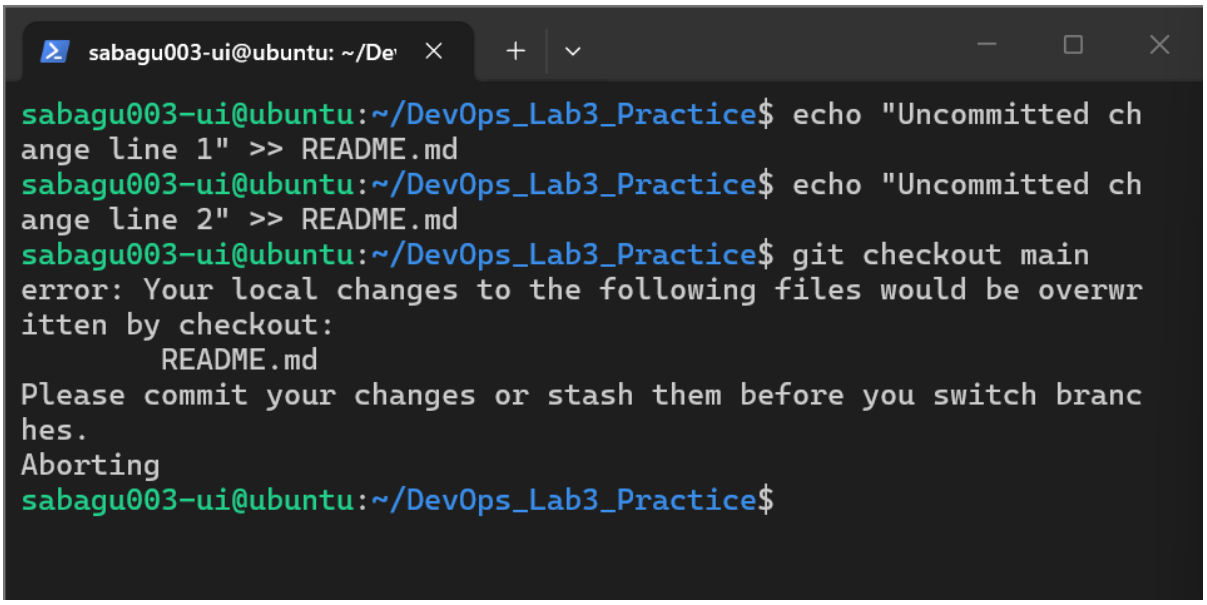



*Figure 4.2: Added uncommitted changes and tried git checkout main (checkout_error.png)*

**Step 3:** Saved the uncommitted changes with git stash.

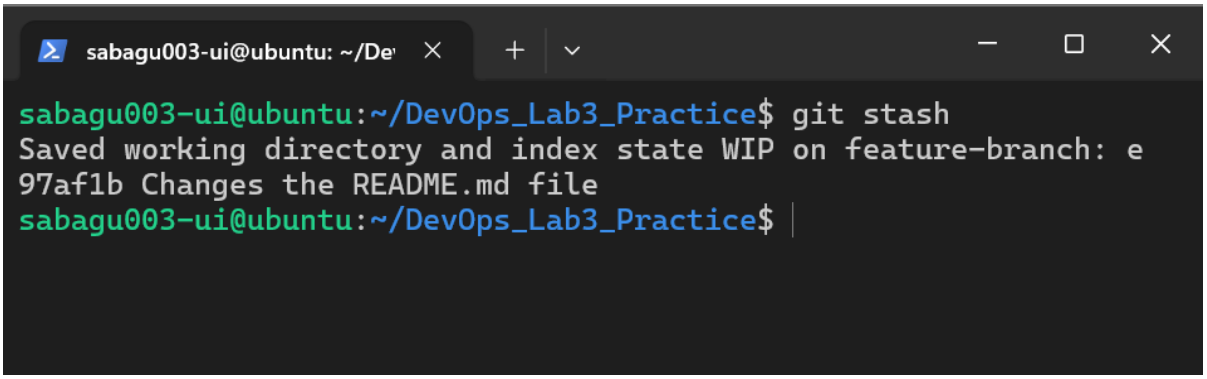



*Figure 4.3: Saved the uncommitted changes with git stash (stash_command.png)*

**Step 4:** Switched to main successfully.

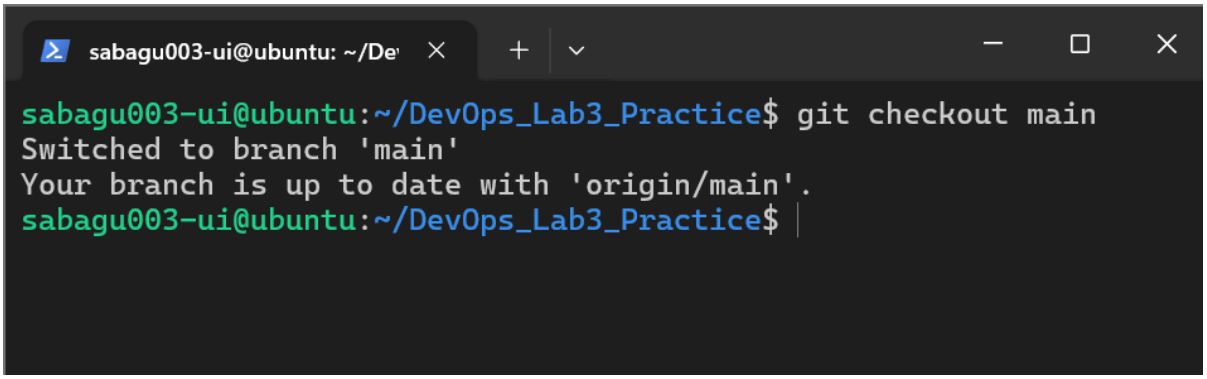



*Figure 4.4: Switched to main successfully (branch_switched.png)*

**Step 5:** Switched back to feature-branch.

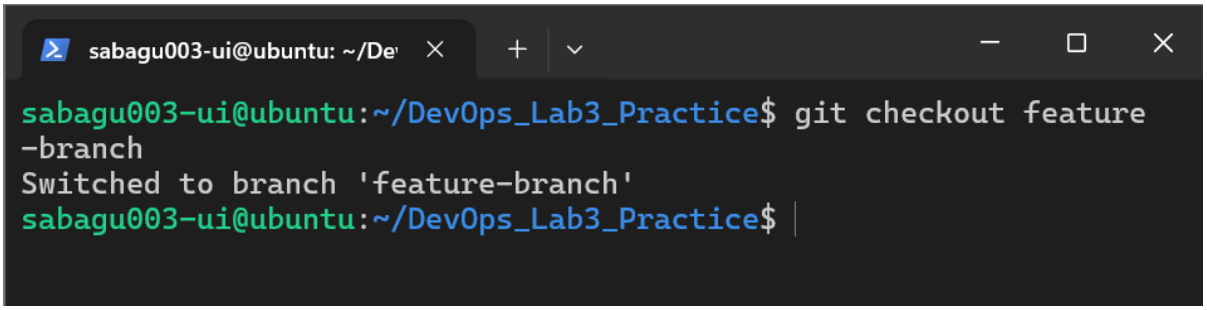



*Figure 4.5: Switched back to feature-branch (back_to_feature.png)*

**Step 6:** git status shows a clean working tree.

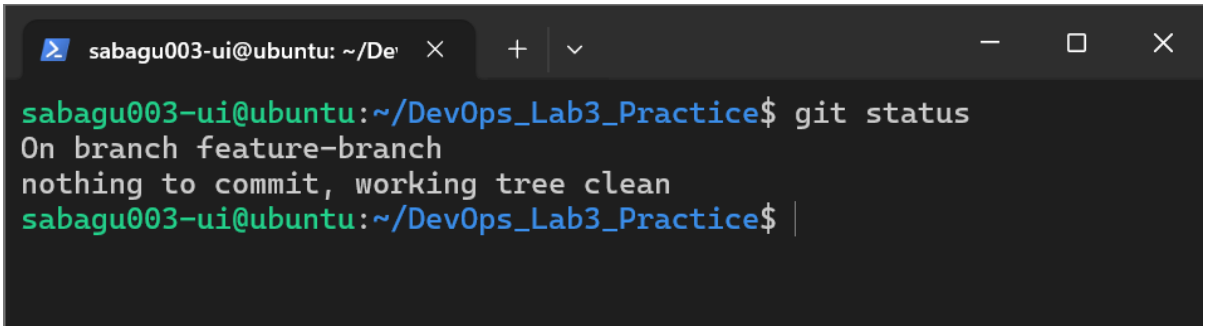



*Figure 4.6: git status shows a clean working tree (status_clean.png)*

**Step 7:** Restored the stashed changes with git stash pop.

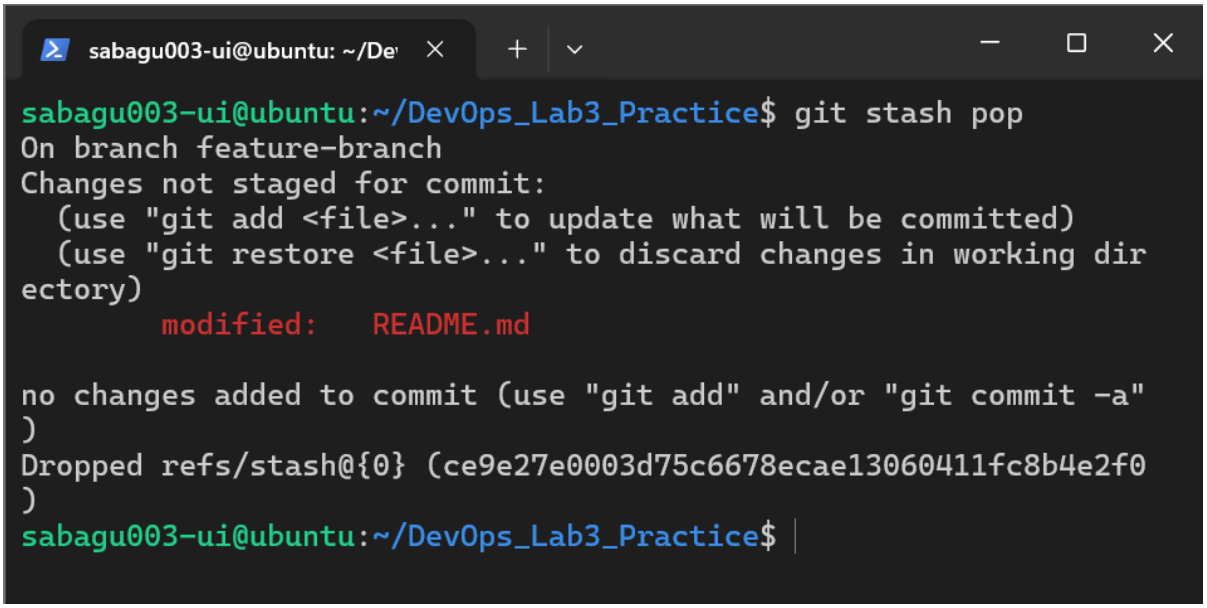



*Figure 4.7: Restored the stashed changes with git stash pop (stash_pop.png)*

## Task 5: Checkout a Specific Commit Using git log

**Step 1:** Viewed the commit history with git log --oneline.

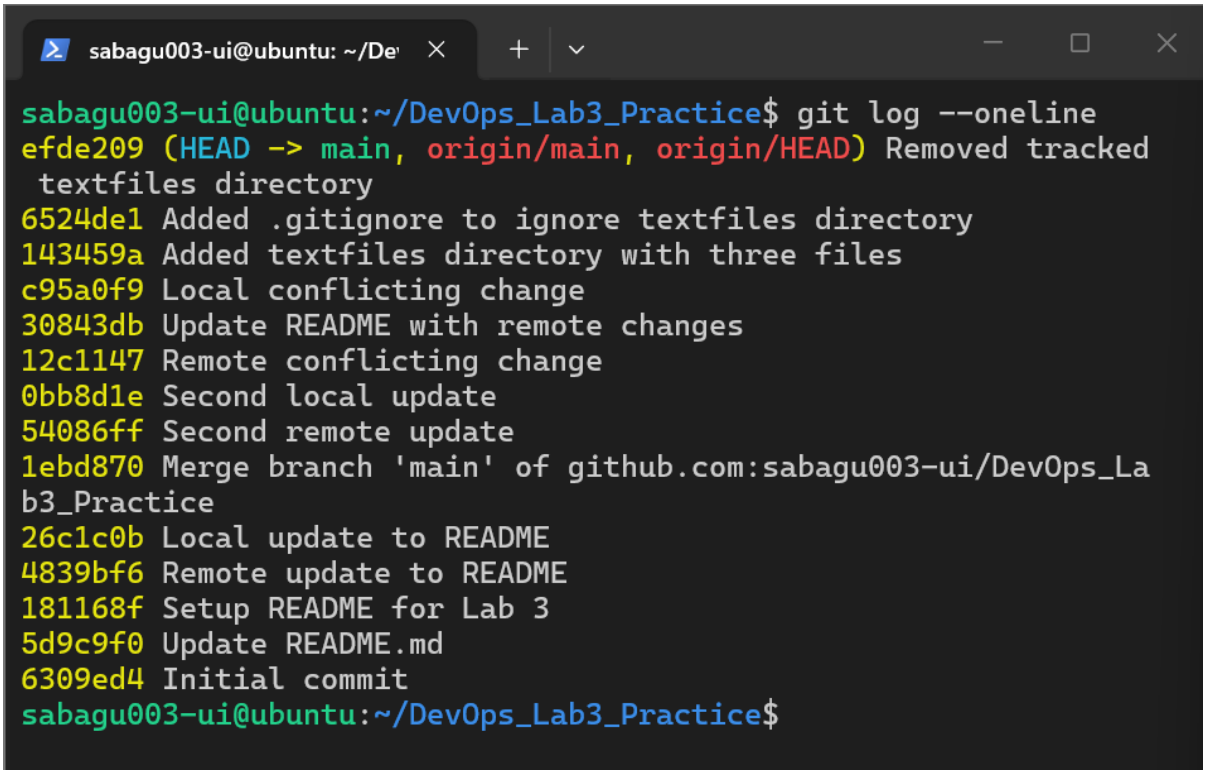



*Figure 5.1: Viewed the commit history with git log --oneline (log_before_checkout.png)*

**Step 2:** Checked out an older commit by its hash. Git shows the detached HEAD state.

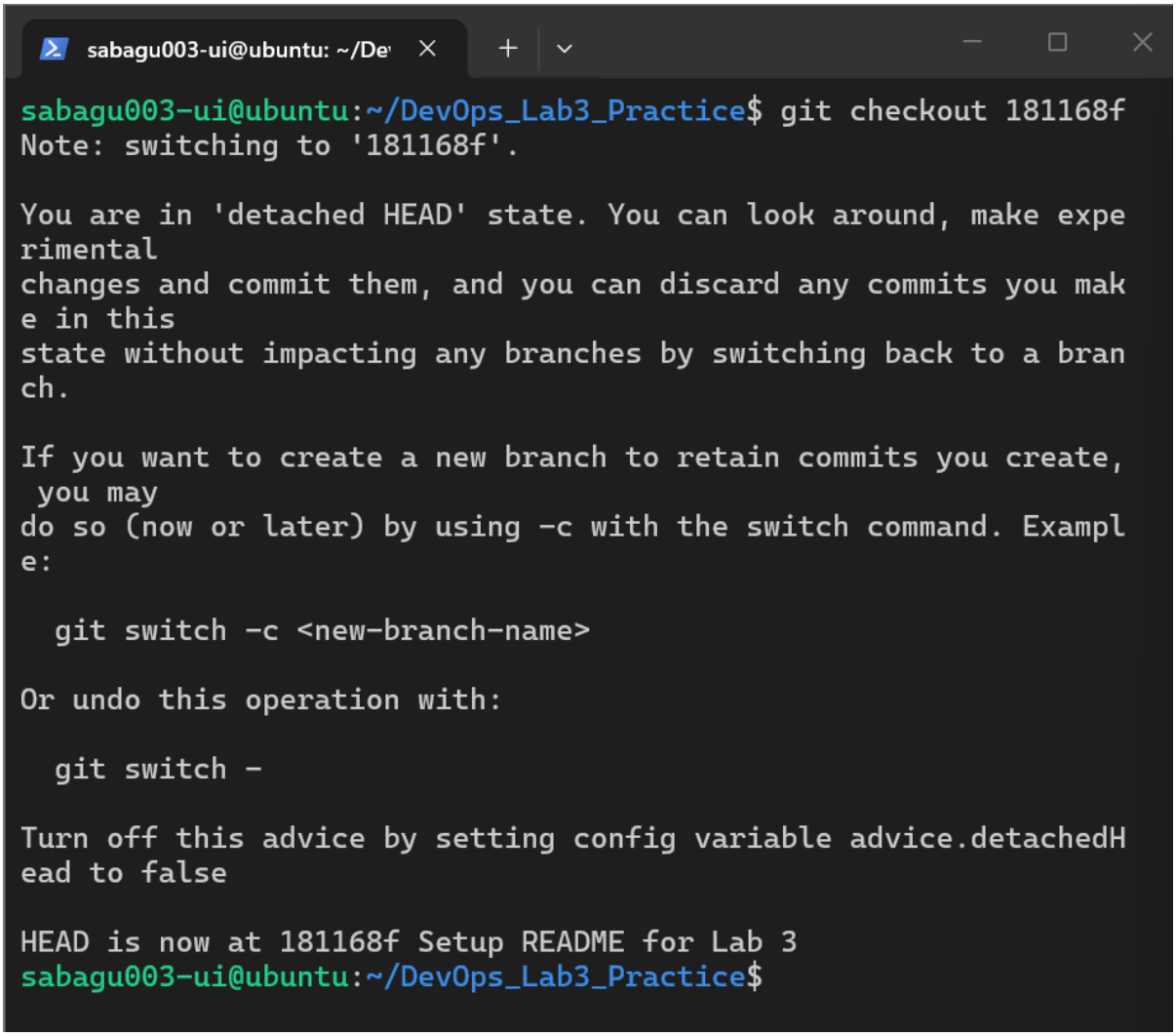



*Figure 5.2: Checked out an older commit by its hash (detached_head.png)*

**Step 3:** Returned to the main branch.

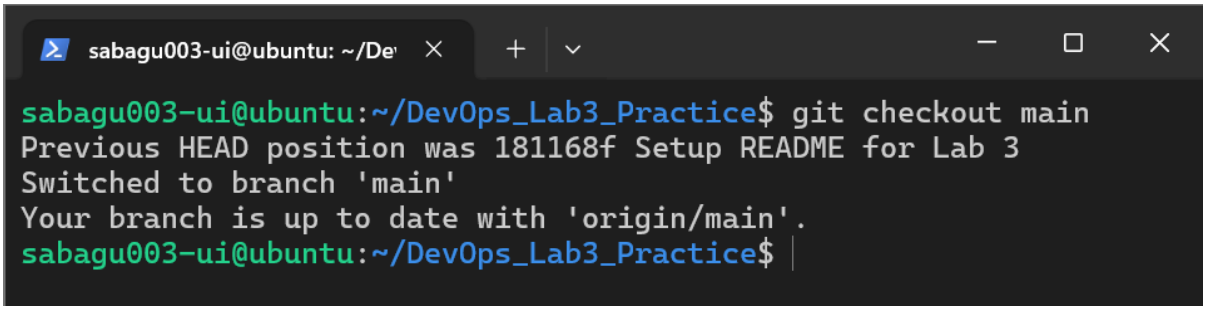



*Figure 5.3: Returned to the main branch (back_to_main.png)*

## Task 6: Resetting Commits (Soft vs Hard Reset)

**Step 1:** Added a line to README.md and committed it (Added test line).

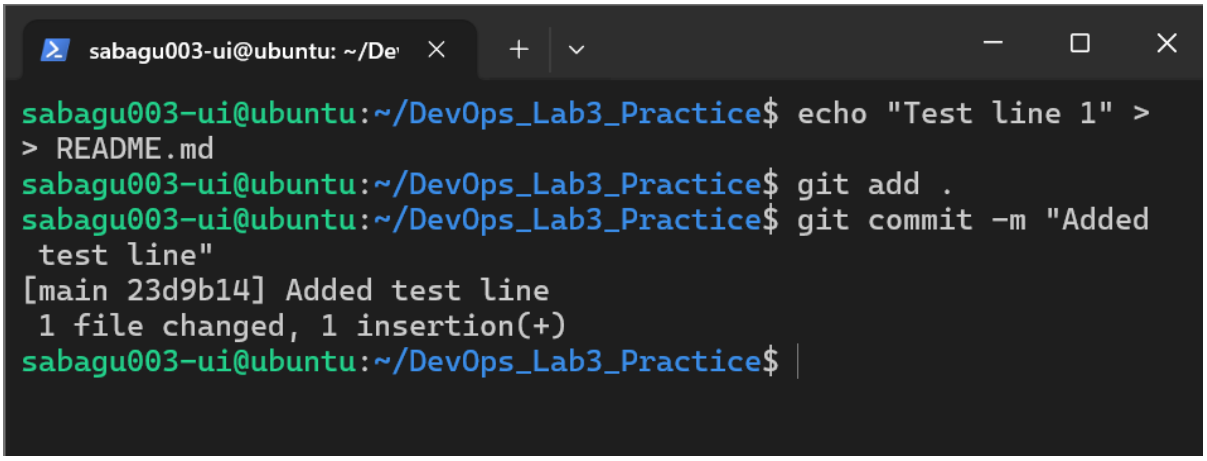



*Figure 6.1: Added a line to README.md and committed it (Added test line) (first_commit.png)*

**Step 2:** Added another line and committed it (Second test commit).

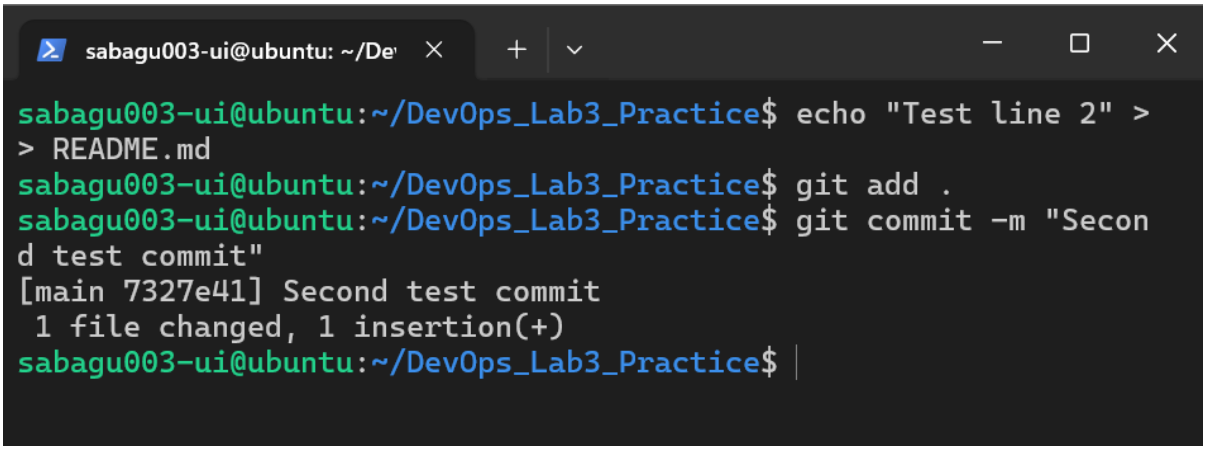



*Figure 6.2: Added another line and committed it (Second test commit) (second_commit.png)*

**Step 3:** Commit history before the reset.

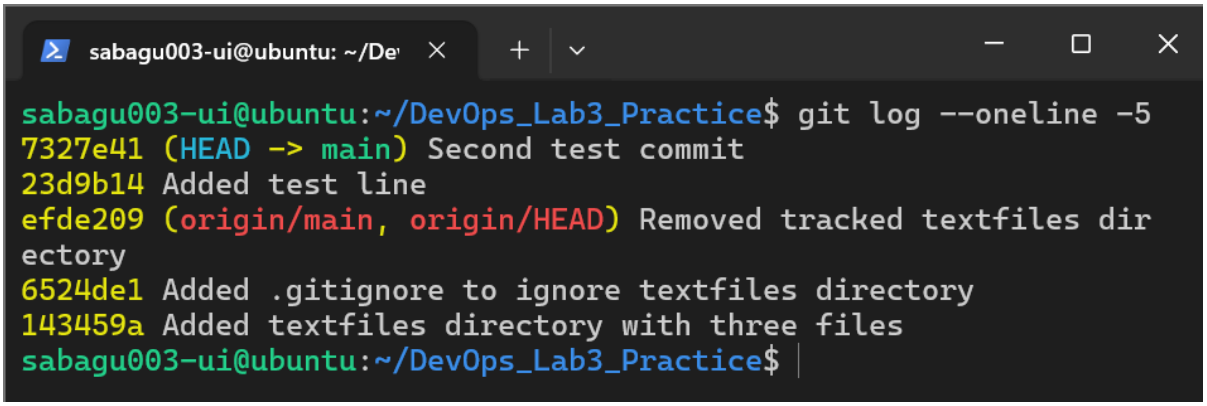



*Figure 6.3: Commit history before the reset (log_before_reset.png)*

**Step 4:** The file contains both added lines before the reset.

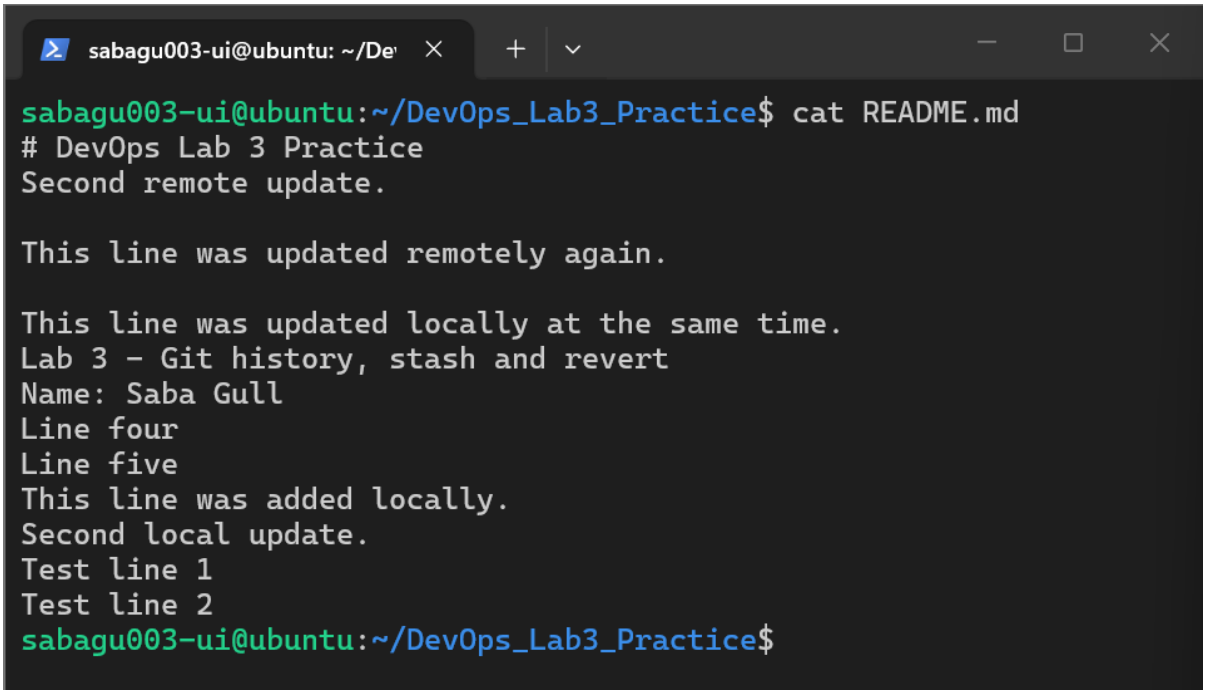



*Figure 6.4: The file contains both added lines before the reset (file_before_reset.png)*

**Step 5:** Performed git reset --soft HEAD~1. The changes stay staged.

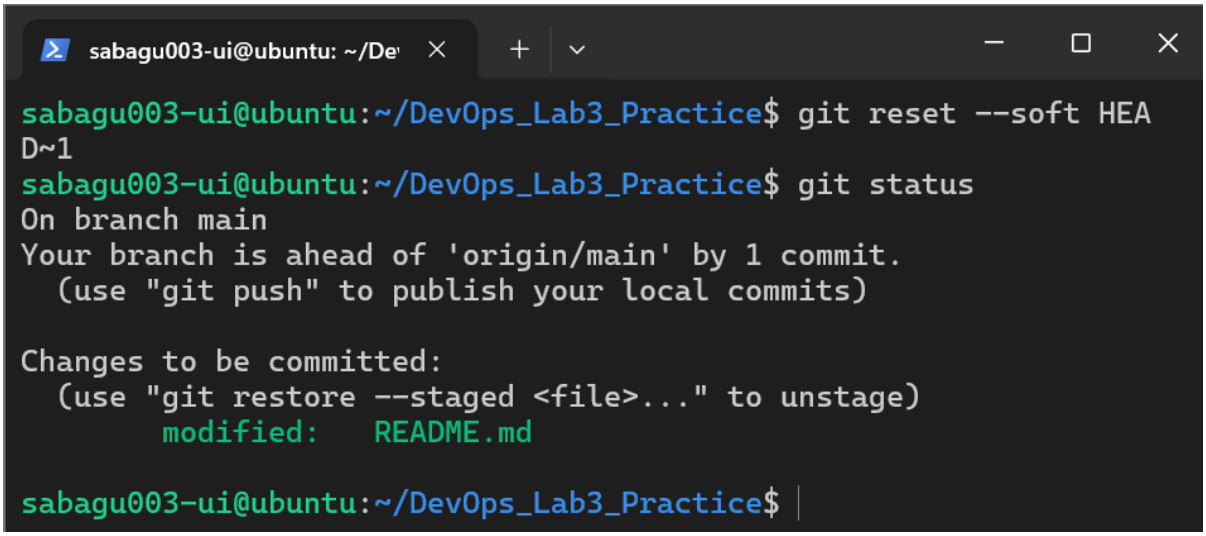



*Figure 6.5: Performed git reset --soft HEAD~1 (soft_reset.png)*

**Step 6:** After the soft reset the last commit is removed from the history.

```
sabagu003-ui@ubuntu:~/DevOps_Lab3_Practice$ git reset --soft HEA
D~1
sabagu003-ui@ubuntu:~/DevOps_Lab3_Practice$ git log --oneline -5
1bfc40a (HEAD -> main, origin/test-force, origin/main, origin/HE
AD, test-force) Revert "Added temporary text file"
d1efc00 Added temporary text file
0acde16 Fix log message
23d9b14 Added test line
efde209 Removed tracked textfiles directory
sabagu003-ui@ubuntu:~/DevOps_Lab3_Practice$
```

*Figure 6.6: After the soft reset the last commit is removed from the history (log_after_soft_reset.png)*

**Step 7:** Both edits are still present in the file and the changes are staged.

```
sabagu003-ui@ubuntu: ~/De
sabagu003-ui@ubuntu:~/DevOps_Lab3_Practice$ cat README.md
# DevOps Lab 3 Practice
Second remote update.

This line was updated remotely again.

This line was updated locally at the same time.
Lab 3 - Git history, stash and revert
Name: Saba Gull
Line four
Line five
This line was added locally.
Second local update.
Test line 1
Test line 2
sabagu003-ui@ubuntu:~/DevOps_Lab3_Practice$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

sabagu003-ui@ubuntu:~/DevOps_Lab3_Practice$
```

*Figure 6.7: Both edits are still present in the file and the changes are staged (file_after_soft_reset.png)*

**Step 8:** Committed the staged changes again (Second Test commit).

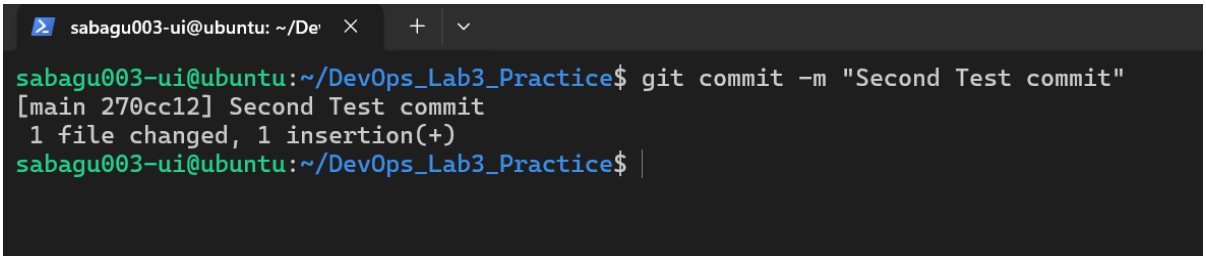



*Figure 6.8: Committed the staged changes again (Second Test commit) (file_after_hard_reset2.png)*

**Step 9:** Performed git reset --hard HEAD~1. The last commit and its changes are discarded.

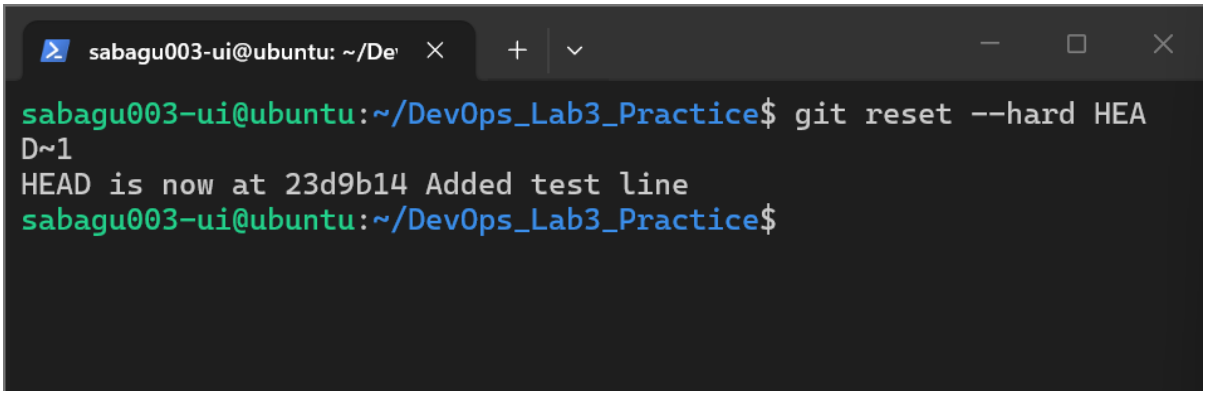



*Figure 6.9: Performed git reset --hard HEAD~1 (hard_reset.png)*

**Step 10:** Commit history after the hard reset.

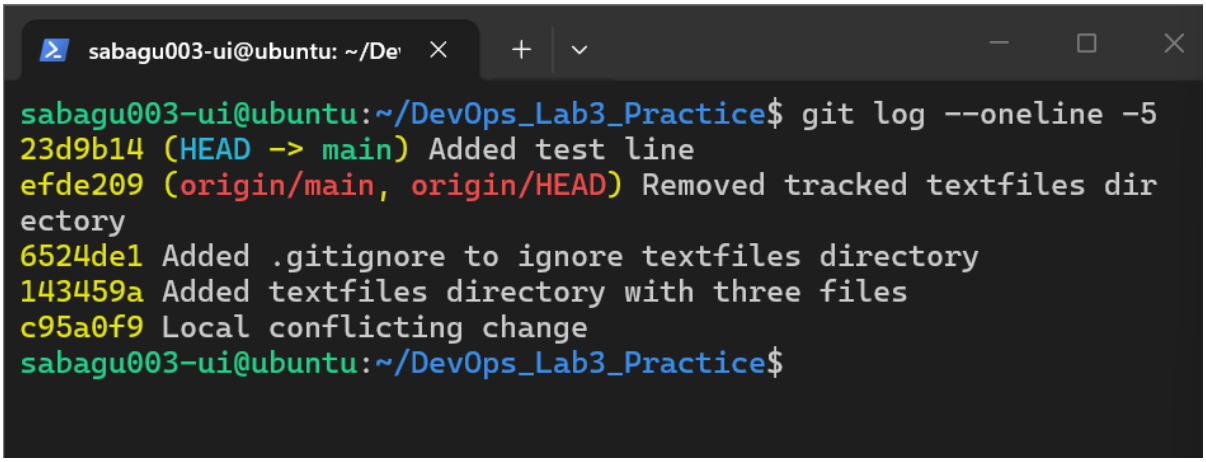



*Figure 6.10: Commit history after the hard reset (log_after_hard_reset.png)*

**Step 11:** The second edit is gone from the file and git status reports a clean working tree.

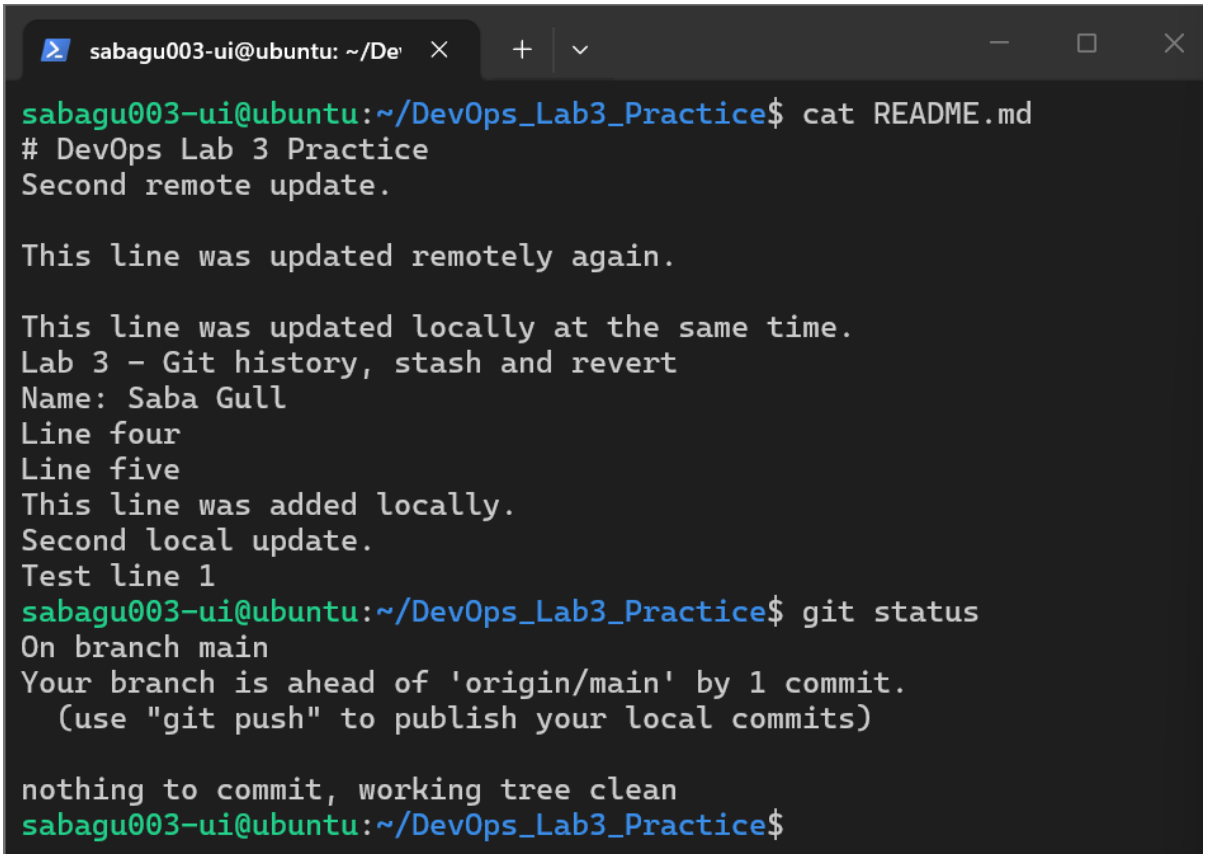



*Figure 6.11: The second edit is gone from the file and git status reports a clean working tree (file_after_hard_reset.png)*

## Task 7: Amending the Last Commit

**Step 1:** Made a change and committed it (Fix log message).

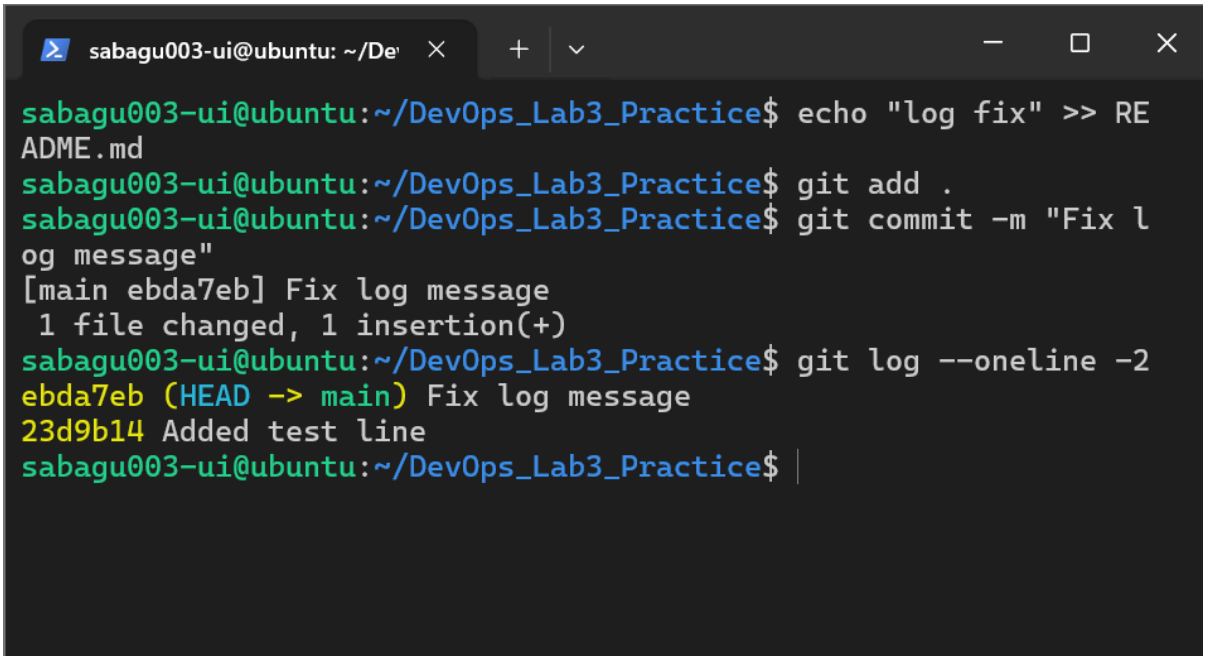



*Figure 7.1: Made a change and committed it (Fix log message) (first_amend_commit.png)*

**Step 2:** Changed another file and amended the last commit with git commit --amend. The single commit now contains both files.

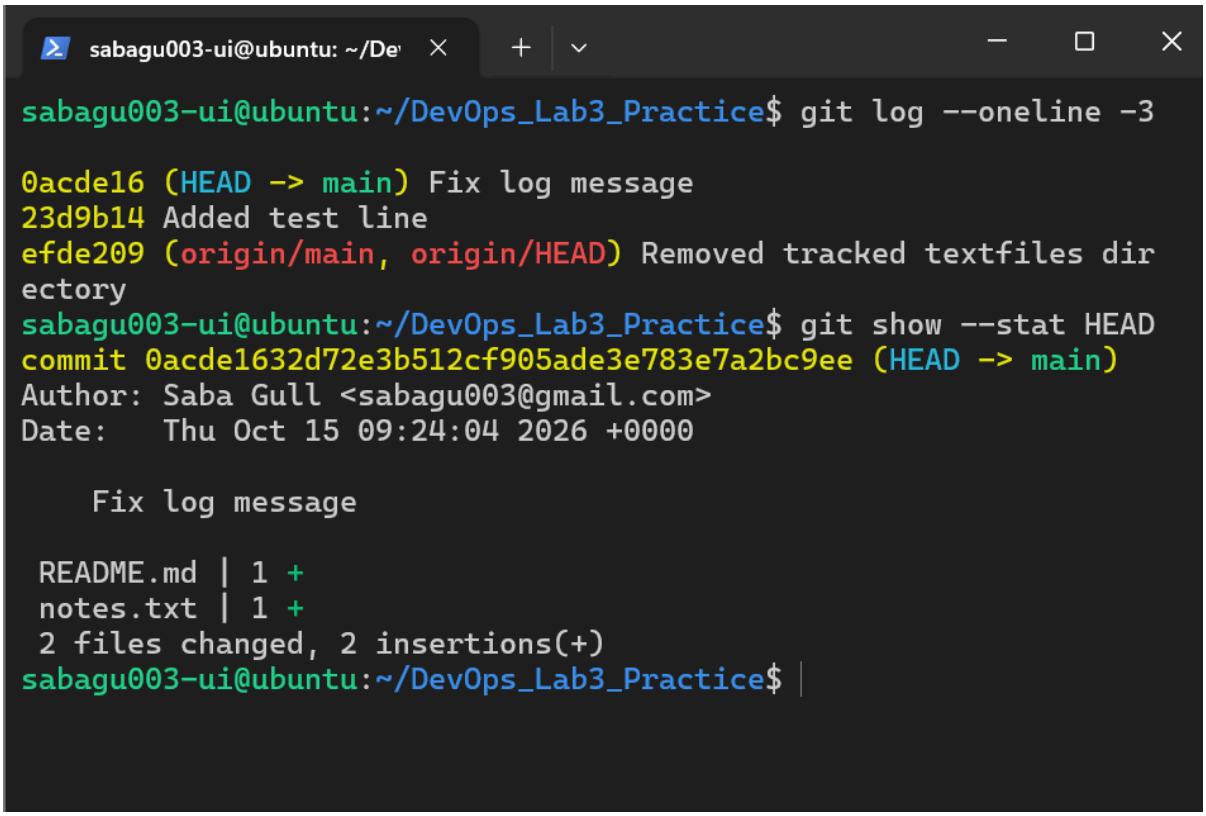



*Figure 7.2: Changed another file and amended the last commit with git commit --amend (amend_commit.png)*

## Task 8: Reverting a Commit (Safe Undo on Remote Branch)

**Step 1:** Created temp.txt, committed it and pushed it to main.

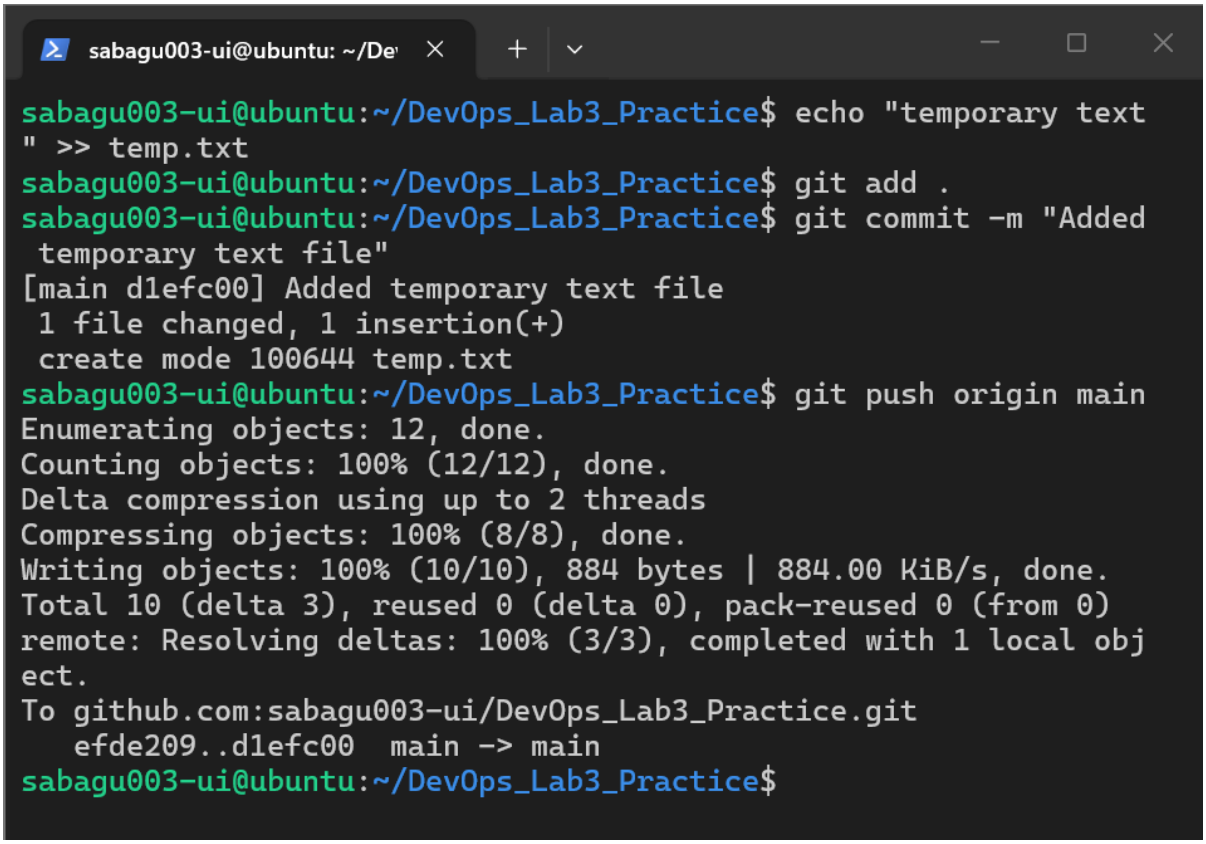



*Figure 8.1: Created temp.txt, committed it and pushed it to main (commit_temp_file.png)*

**Step 2:** Reverted the commit with git revert. A new revert commit undoes the change without rewriting history.

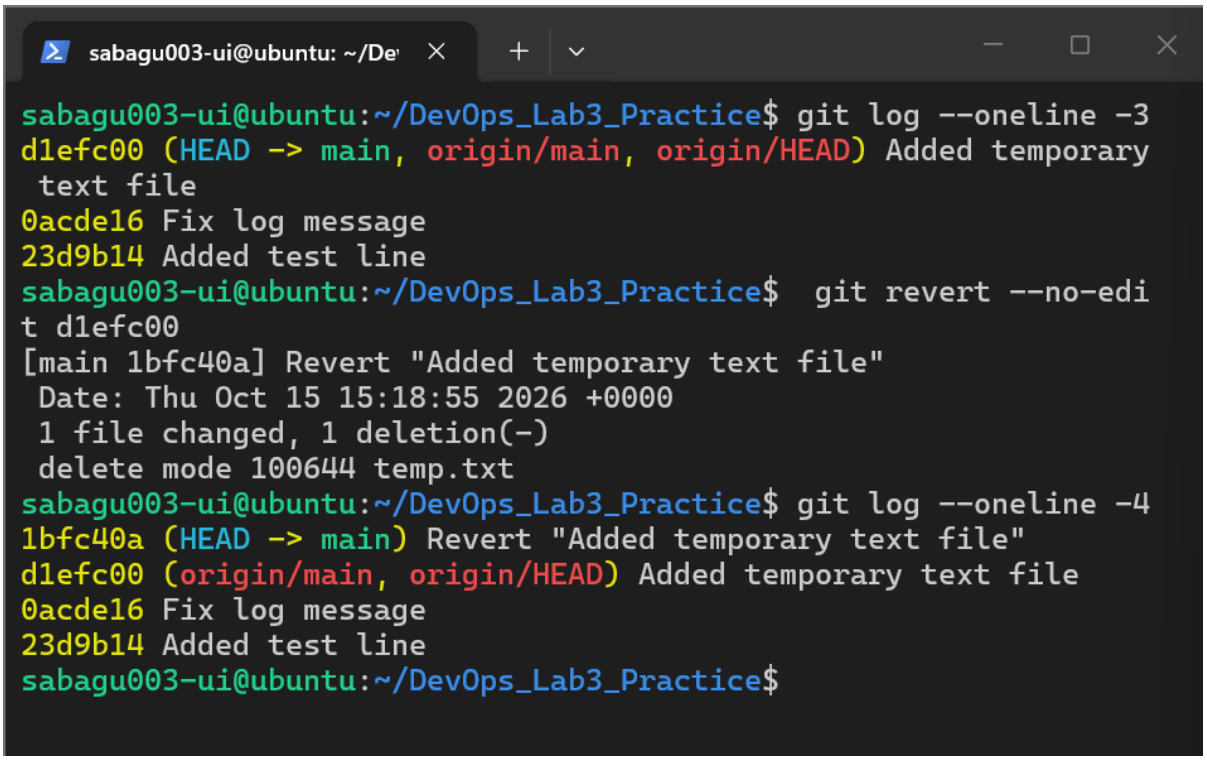



*Figure 8.2: Reverted the commit with git revert (revert_commit.png)*

**Step 3:** Pushed the revert commit to GitHub.

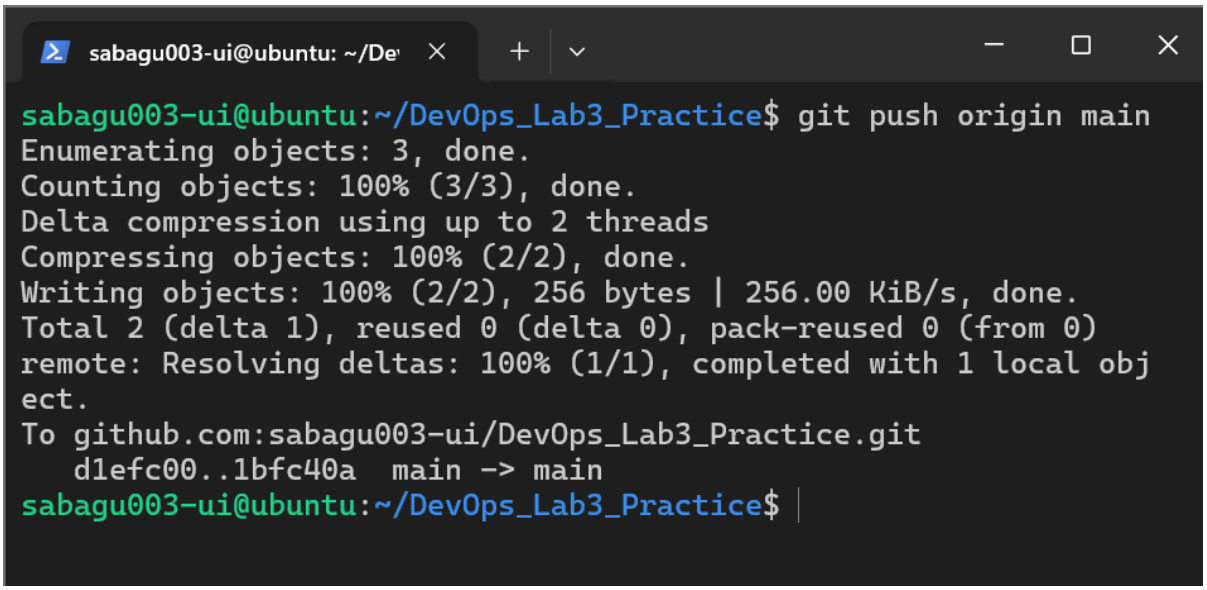



*Figure 8.3: Pushed the revert commit to GitHub (revert_push.png)*

## Task 9: Force Push (With Caution)

**Step 1:** Created the branch test-force.

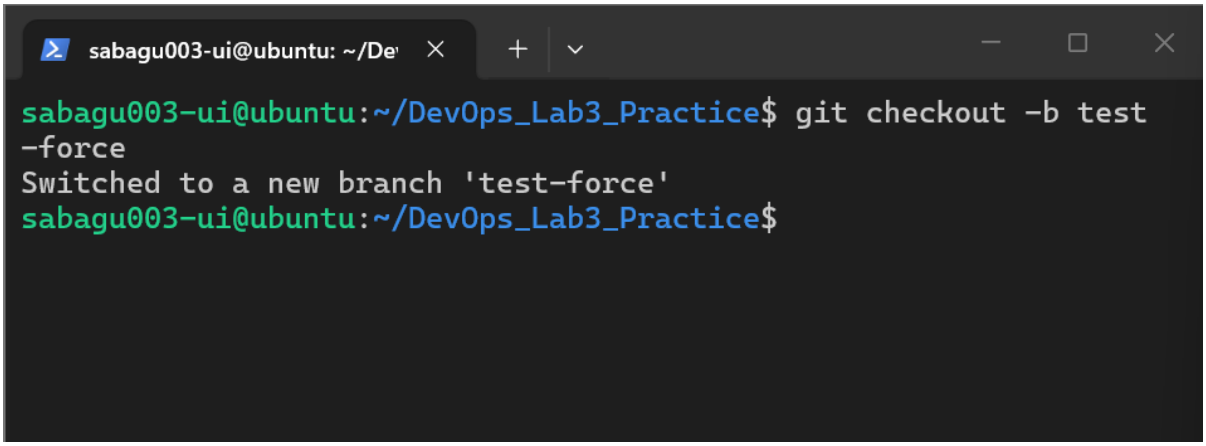



*Figure 9.1: Created the branch test-force (new_branch.png)*

**Step 2:** Made a small change and committed it on test-force.

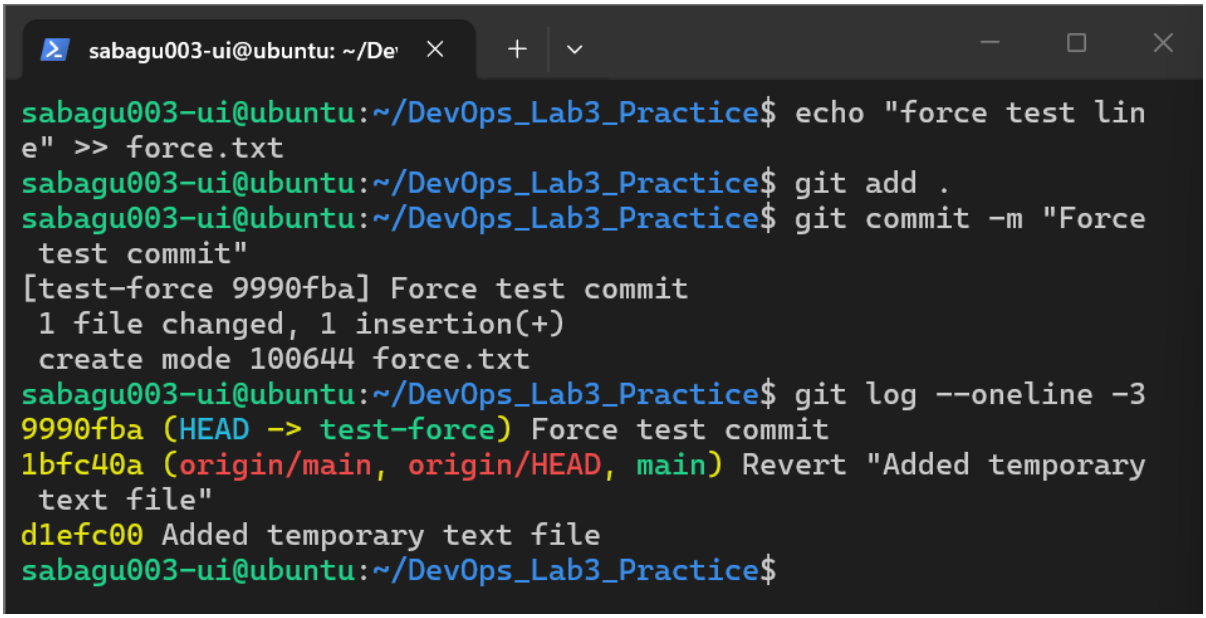



*Figure 9.2: Made a small change and committed it on test-force (force_commit.png)*

**Step 3:** Pushed test-force to GitHub.

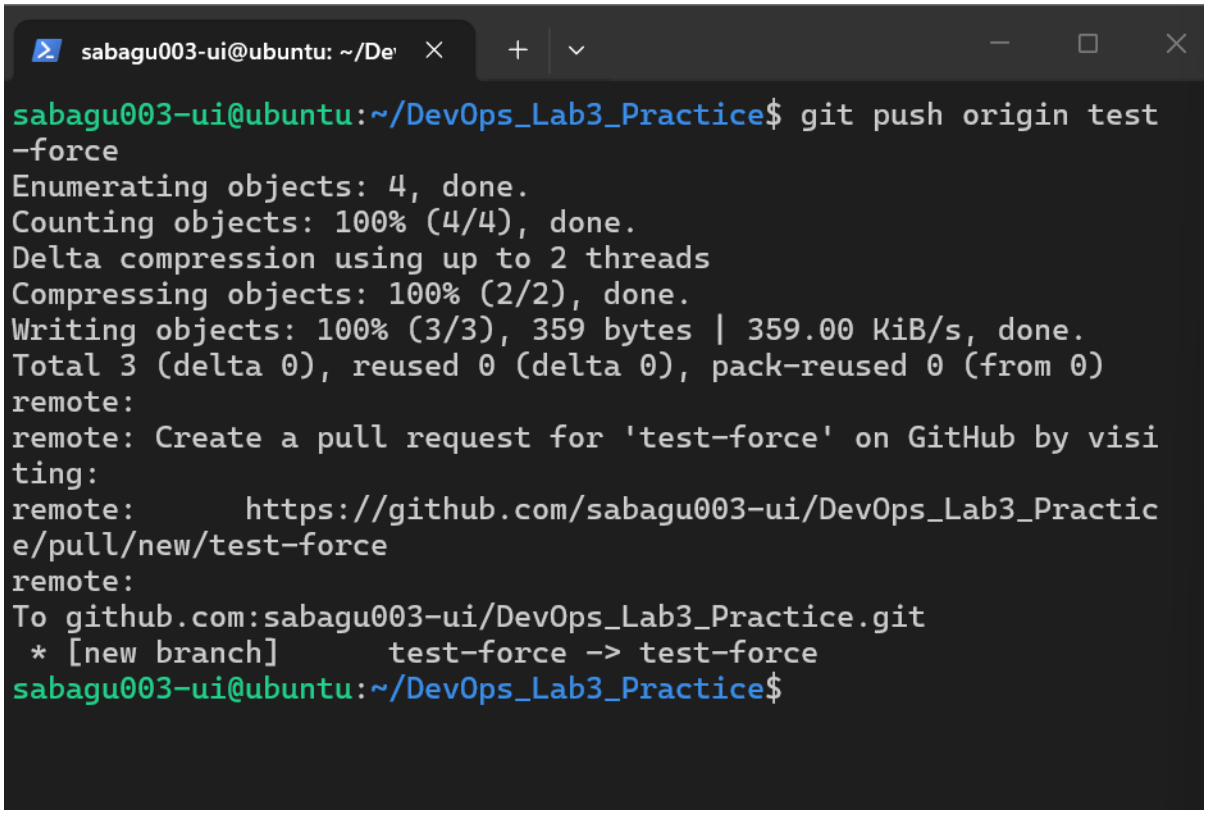



*Figure 9.3: Pushed test-force to GitHub (push_force_branch.png)*

**Step 4:** Performed git reset --hard HEAD~1 on test-force.

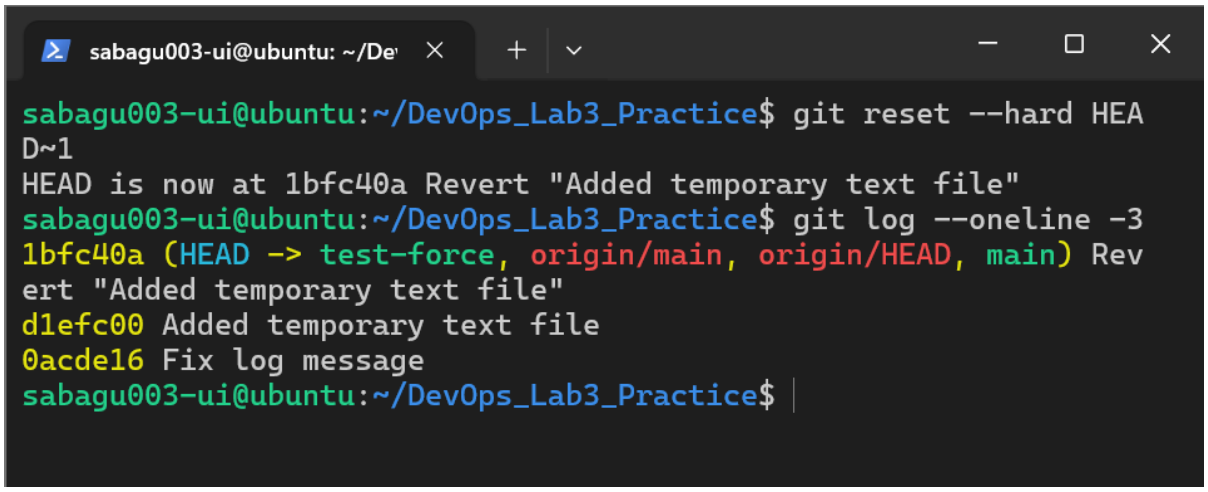



*Figure 9.4: Performed git reset --hard HEAD~1 on test-force (hard_reset_force.png)*

**Step 5:** A normal push was rejected by the remote (non-fast-forward).

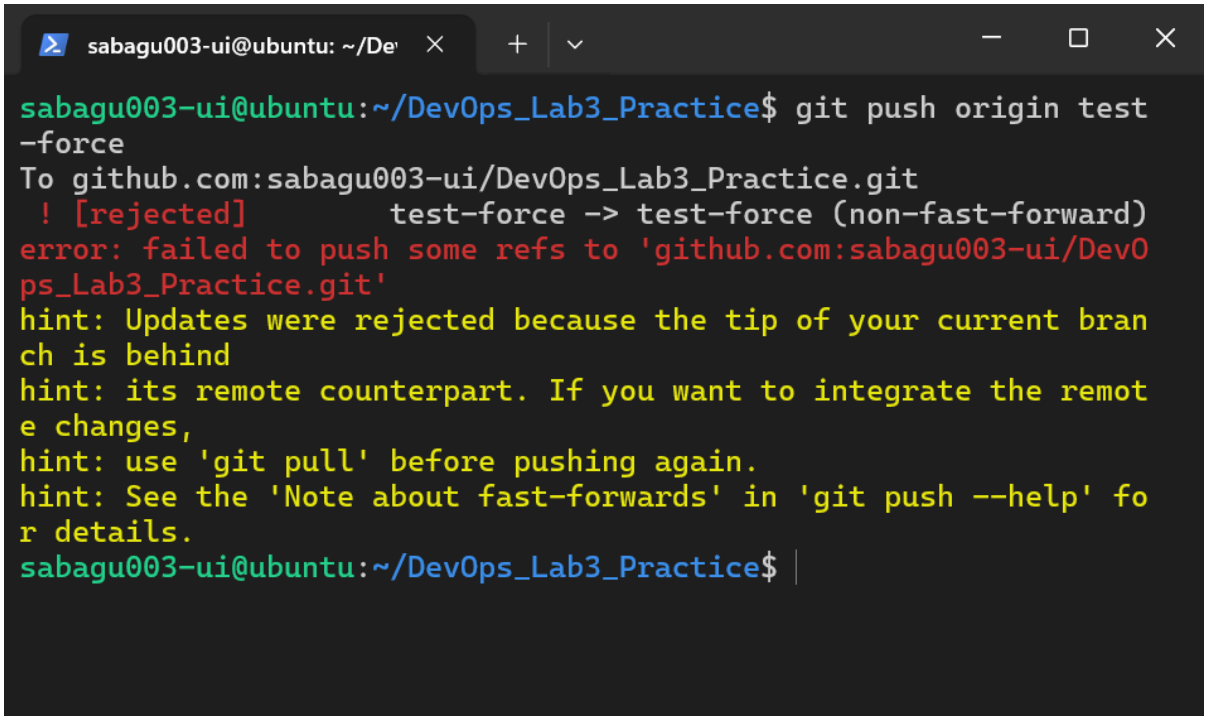



*Figure 9.5: A normal push was rejected by the remote (non-fast-forward) (normal_push.png)*

**Step 6:** Force-pushed with git push origin test-force --force. The remote branch history was rewritten.

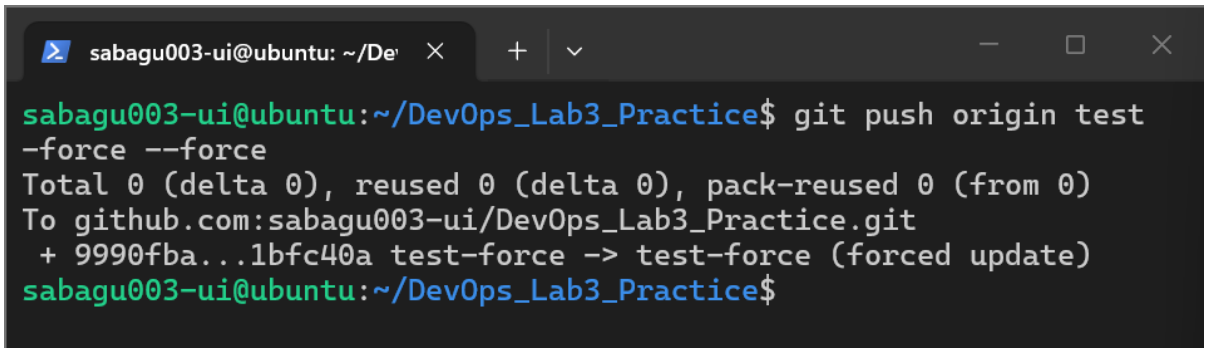



*Figure 9.6: Force-pushed with git push origin test-force --force (force_push.png)*

## Task 10: Running Gitea in GitHub Codespaces via Docker Compose

**Step 1:** Forked the Gitea repository to my GitHub account.

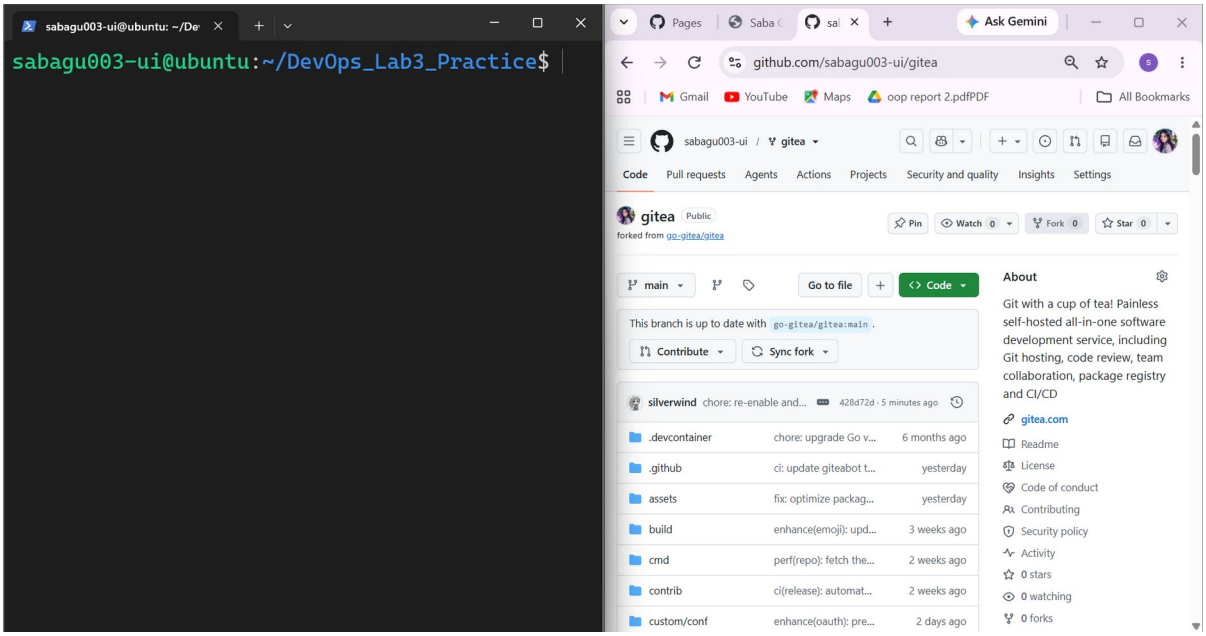



*Figure 10.1: Forked the Gitea repository to my GitHub account (forked_gitea.png)*

**Step 2:** Created a Codespace and it is loading.

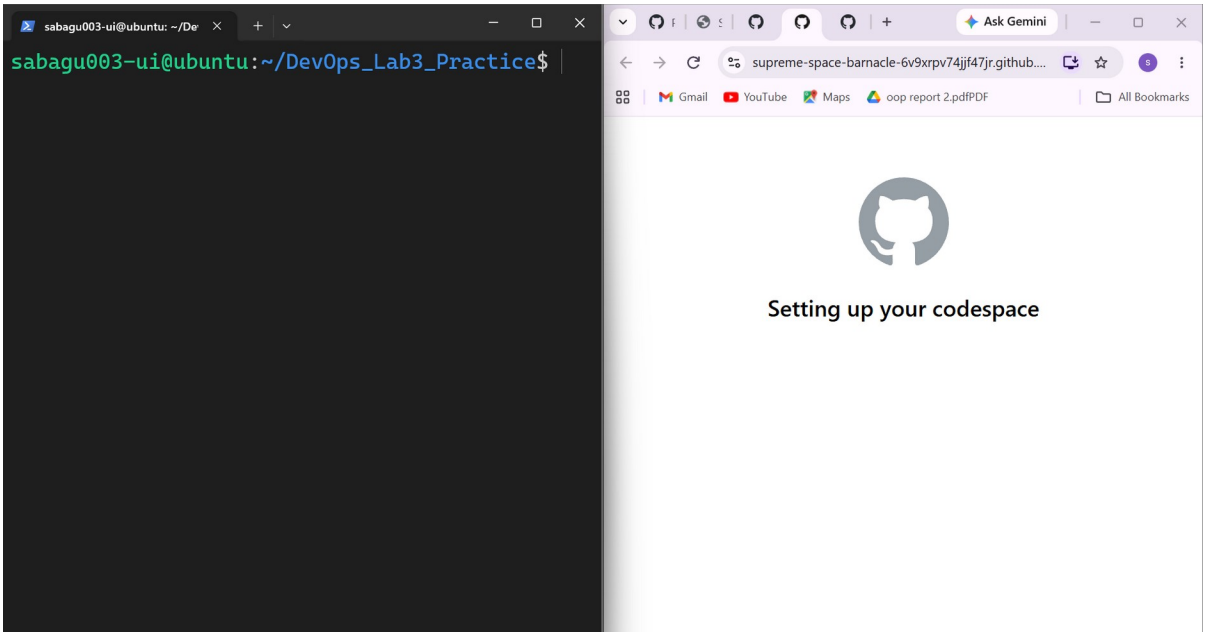



*Figure 10.2: Created a Codespace and it is loading (codespace_loading.png)*

**Step 3:** Started Gitea with docker compose up -d. docker compose ps shows the container running.

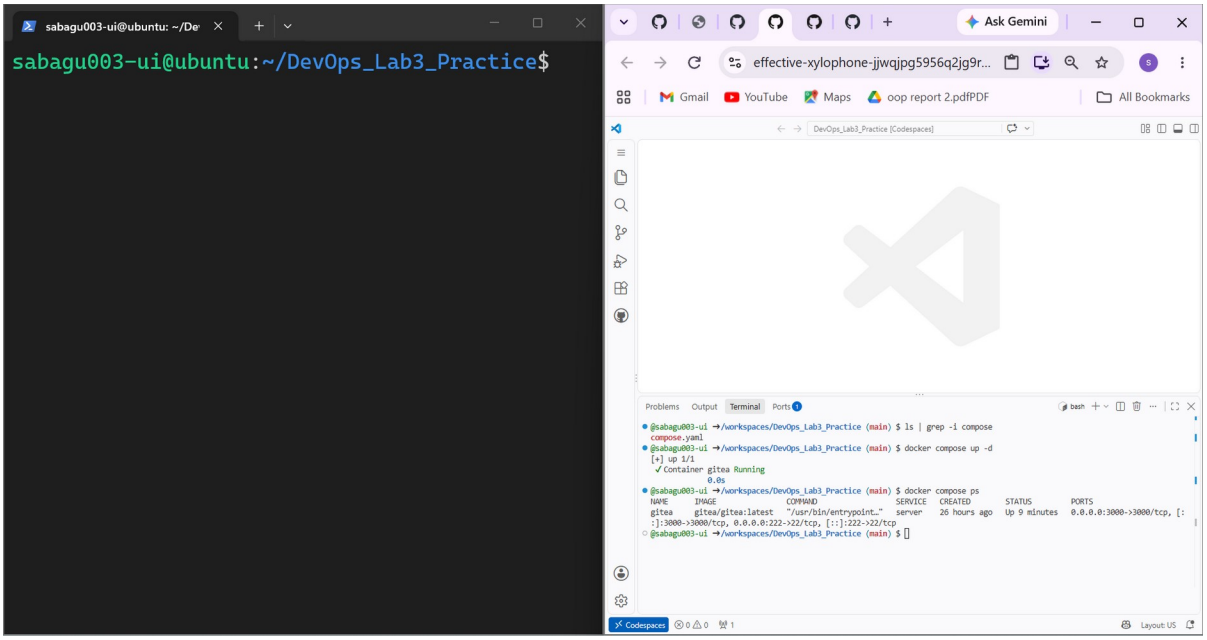



*Figure 10.3: Started Gitea with docker compose up -d (docker_up.png)*

**Step 4:** Opened forwarded port 3000 (public). The Gitea installation page is shown.

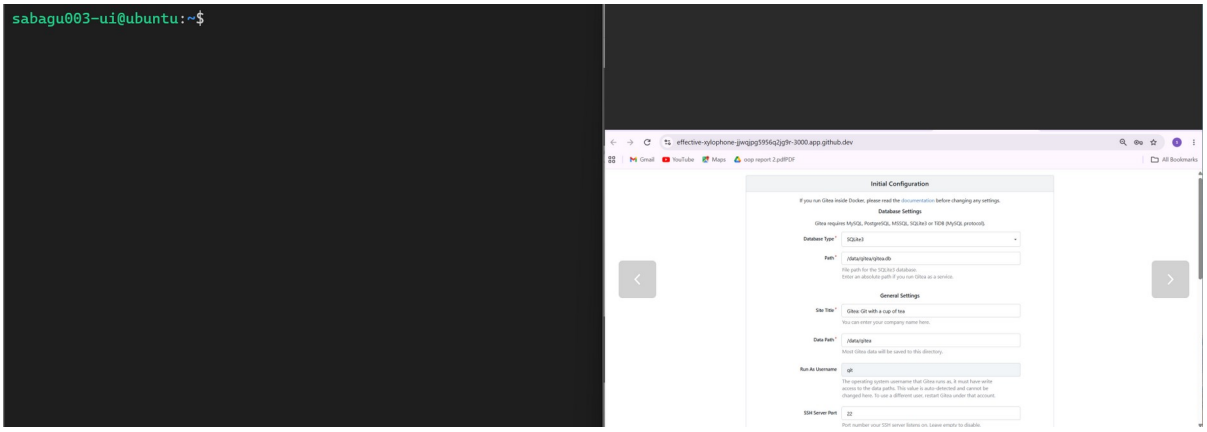



*Figure 10.4: Opened forwarded port 3000 (public) (gitea_install_page.png)*

**Step 5:** Filled in the installation form including the administrator account.

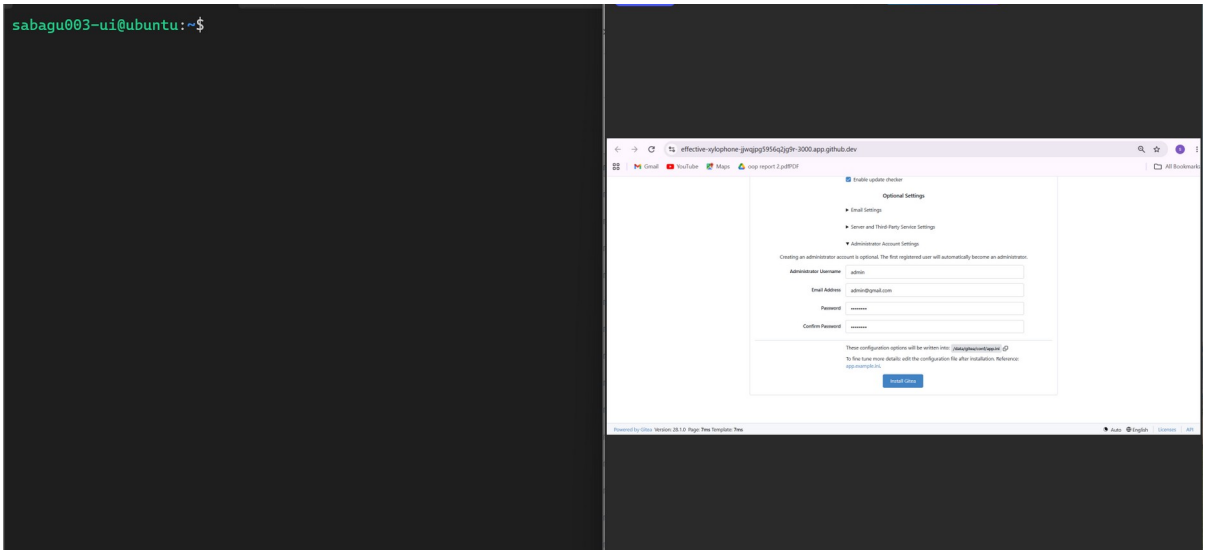



*Figure 10.5: Filled in the installation form including the administrator account (admin_setup.png)*

**Step 6:** Logged in to Gitea. The dashboard is shown.

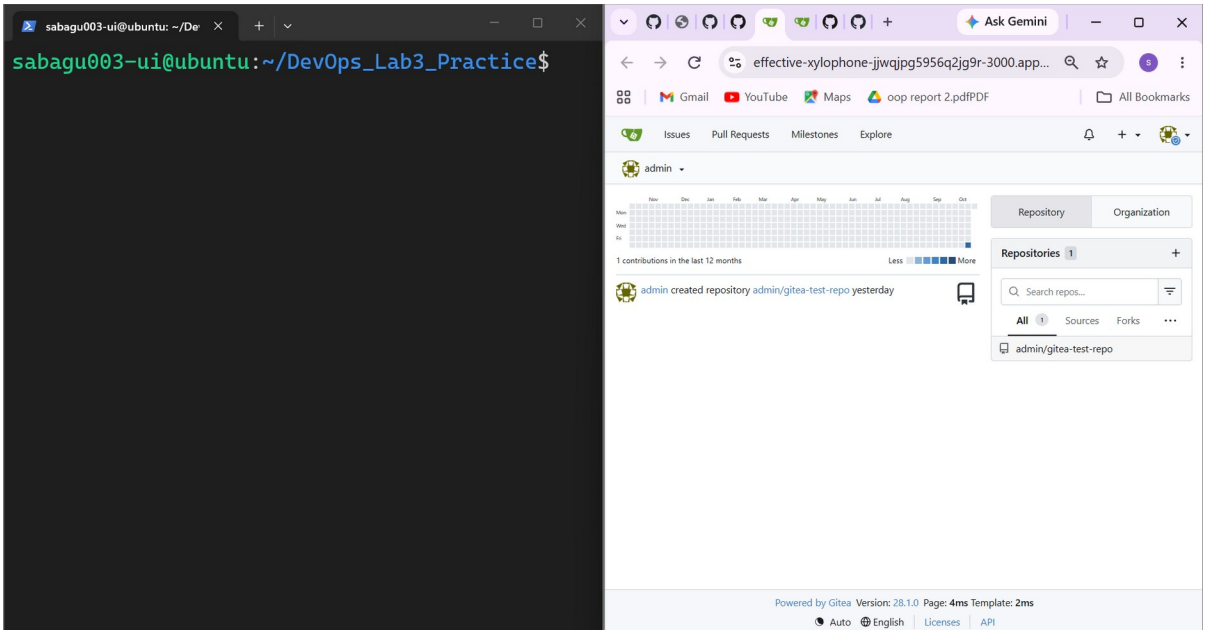



*Figure 10.6: Logged in to Gitea (gitea_dashboard.png)*

**Step 7:** Created a new repository (gitea-test-repo) in Gitea.

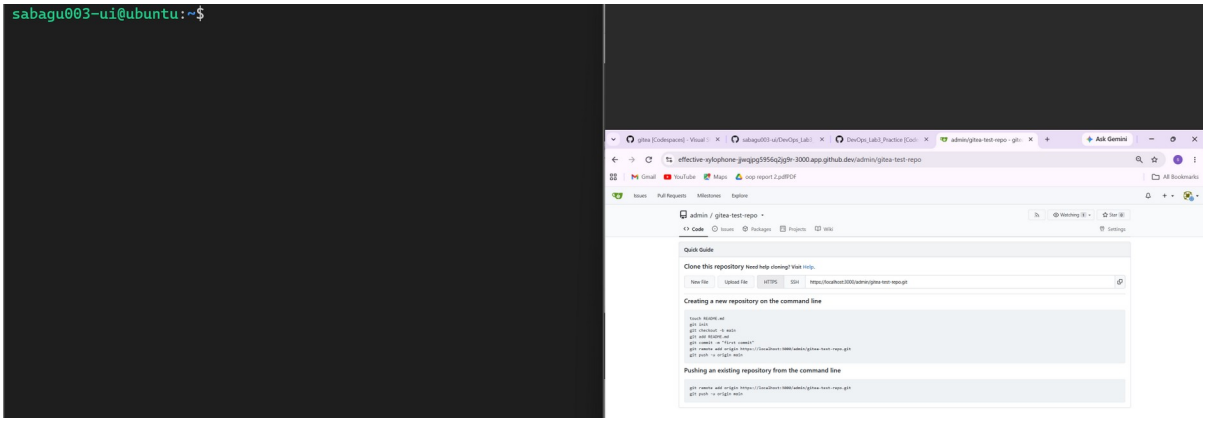



*Figure 10.7: Created a new repository (gitea-test-repo) in Gitea (gitea_new_repo.png)*

## Task 11: Creating a GitHub Pages Portfolio Site

**Step 1:** Created the public repository sabagu003-ui.github.io.

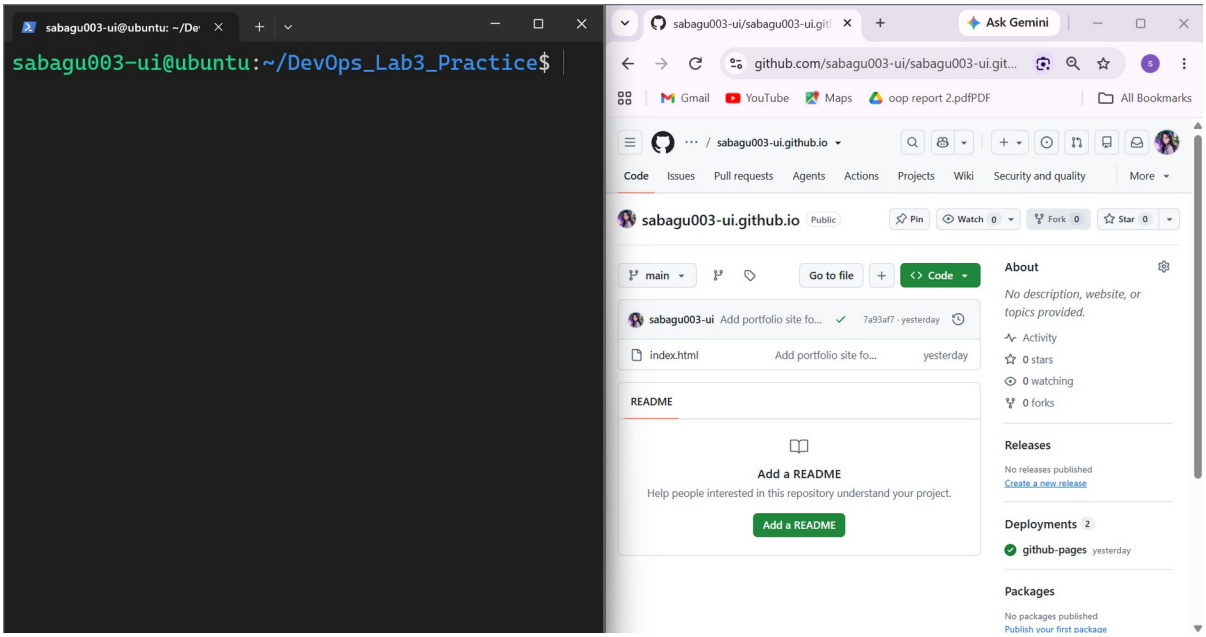



*Figure 11.1: Created the public repository sabagu003-ui.github.io (github_pages_repo.png)*

**Step 2:** Local folder containing the portfolio website (index.html).

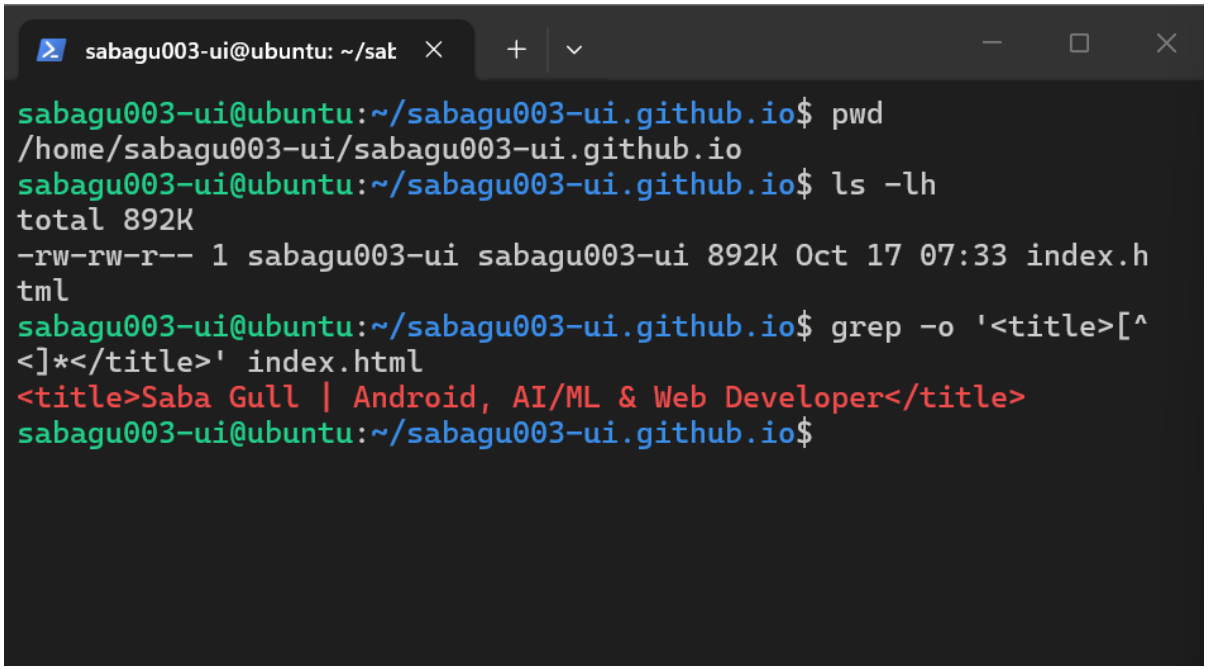



*Figure 11.2: Local folder containing the portfolio website (index.html) (local_static_site.png)*

**Step 3:** Initialized git, added the remote, committed and pushed the files to GitHub.

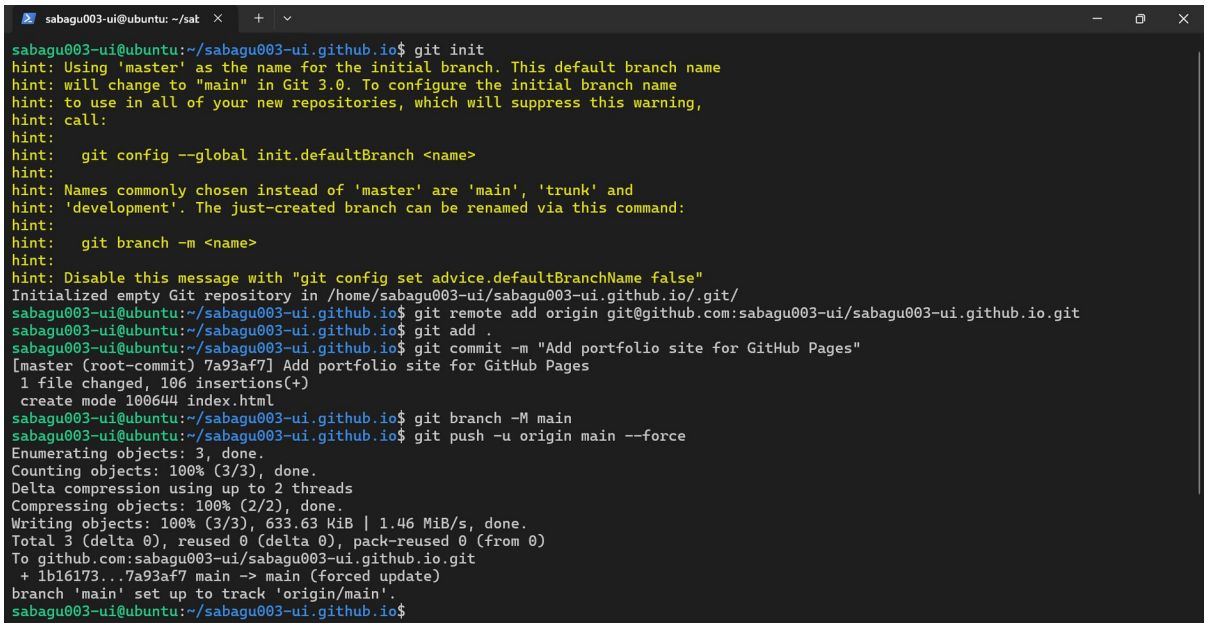



*Figure 11.3: Initialized git, added the remote, committed and pushed the files to GitHub (push_static_site.png)*

**Step 4:** GitHub Pages settings: the site is published from the main branch.

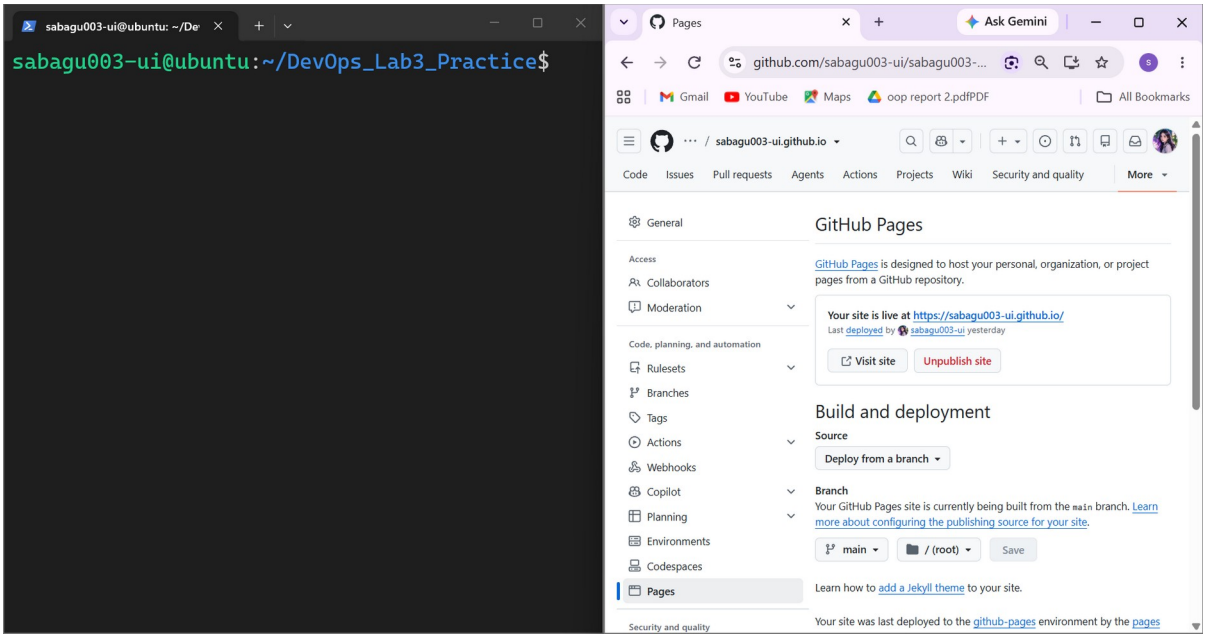



*Figure 11.4: GitHub Pages settings: the site is published from the main branch (github_pages_settings.png)*

**Step 5:** The live portfolio site at https://sabagu003-ui.github.io.

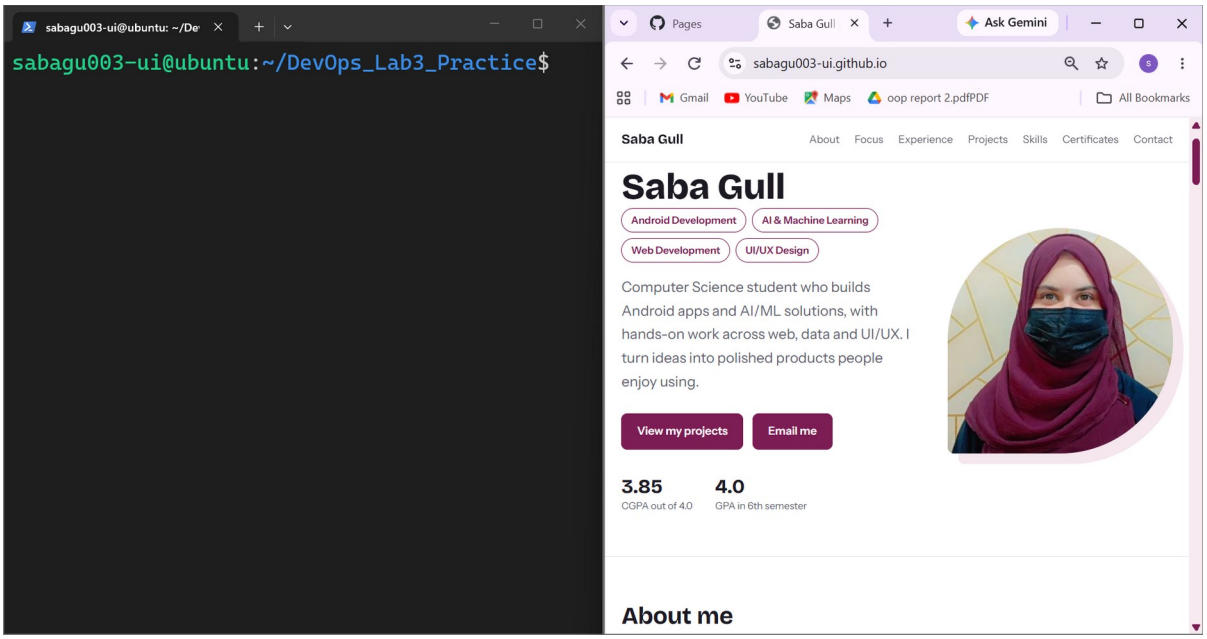



*Figure 11.5: The live portfolio site at https://sabagu003-ui.github.io (live_site.png)*